

CAMP-VS 虚拟染色竞赛项目技术报告

从 DAPI 图像到 CD68 虚拟染色的完整工程解析

基于当前项目源码、配置与本地官方数据审计结果整理

源码核对日期：2026 年 8 月 24 日

摘要

本文面向第一次接触医学图像、深度学习和命令行的读者，完整说明 CAMP-VS 虚拟染色竞赛工程。项目把一张 DAPI 荧光显微镜图像作为输入，预测同一位置的 CD68 标记图像。本文从生物学背景讲起，逐步解释数据目录、ROI 分组、manifest、PyTorch DataLoader、Sobel 结构提示、四尺度 NAF 编解码器、余弦原型混合、任务适配器、重型四尺度 Detail U-Net、自适应空间-频率路线、三路内容感知融合、真实 3×3 ROI 邻域、两阶段复合损失、混合精度训练、EMA、三域验证、D4 测试时增强和提交 ZIP 的产生过程。

报告不是根据模型名称猜测而写，而是逐项核对了当前工作区中的 README.md、docs/essay/report.tex、默认/初赛/v2/v3/v4/resolved-local YAML 配置、CLI、训练器、验证器、模型、损失、数据集、指标及提交代码，并读取了本地官方数据审计结果。因此，本文会特别指出三项容易被误解的事实：第一，本地 JPG 在磁盘上是 RGB，只是三个通道逐像素完全相同；第二，configs/resolved_local.yaml 已把这种审计结论解析为 1 个逻辑输入/输出通道，但提交时仍恢复为原始 RGB 存储语义；第三，本地 JPG round-trip 指标是接近最终文件形态的代理指标，但没有官方评测器源码时，不能宣称它与平台算法完全相同。

阅读方法。如果你只学过计算机专业的机器学习和深度学习课程，请不要跳过第 1 节。这一节先建立“组织-ROI-patch-细胞-像素”的共同语言，再解释五个 marker、荧光强度、监督标签和评价层级；理解这些概念后，后面的网络结构和损失函数才不只是公式。如果你只想在 AutoDL 上跑通项目，可直接阅读附录“AutoDL 从上传到训练的保姆级流程”。文中“稳定对照”指 configs/initial_round_cd68_retrain_v2.yaml；“v3”是已停止的轻量多分支候选；“当前候选”或“v4”专指 configs/initial_round_cd68_max_v4.yaml 与默认/resolved-local 配置合并后的实际设置。

目录

1 引言与项目背景	7
1.1 什么是染色，为什么显微镜图像需要它	7

目录	2
1.2 H&E、IHC、免疫荧光不是同一种数据	7
1.3 虚拟染色是什么	8
1.4 DAPI 与 CD68 分别是什么	8
1.5 荧光显微镜图像与 patch	8
1.6 竞赛目标与图像到图像翻译	9
1.7 先建立空间层级：组织、ROI、patch、细胞和像素	9
1.8 一个细胞里有哪些结构，marker 为什么出现在不同位置	10
1.9 本项目五个通道的医学含义	10
1.10 把医学概念放回一张真实的五通道 patch	11
1.11 本项目的 label 到底是什么	13
1.12 配对与配准：同名文件为什么重要	14
1.13 从光子到 Tensor：计算机实际接收了什么	15
1.14 模型学的是条件预测，不是把不可见信息恢复出来	15
1.15 像素级、细胞级、patch 级与 ROI 级评价	16
1.16 SSIM、PSNR 与医学意义应该怎样对应	16
1.17 读后续网络章节前应掌握的最小知识清单	17
2 命令行入门：为什么用 CLI 而不是 python train.py	17
2.1 CLI 是什么	17
2.2 把命令逐段拆开	18
2.3 为什么不是直接运行 train.py	19
2.4 从入口到训练器：autodl-run 内部流程	19
2.5 autodl-submit 内部流程	20
3 数据集格式与加载	20
3.1 本地官方数据的真实目录与数量	20
3.2 真实样本显示了哪些数据分布特征	21
3.3 “RGB 文件”和“灰度内容”不是同一概念	22
3.4 文件名、ROI 与二维坐标	22
3.5 manifest 是什么	23
3.6 按 ROI 划分训练集与验证集	23
3.7 Dataset 与 DataLoader 如何把 JPG 送入模型	23
3.8 PairedTransform 为什么必须“成对”	24
4 模型架构详解	24
4.1 先建立整体直觉：这是一个“先压缩理解，再放大还原”的网络	24
4.2 历史 v3 对照：轻量 U、固定频率与稳定输出（当前未启用）	25
4.3 v4 重型三路主线：为什么替换轻量 U，却不删除旧模型	26
4.4 输入预处理与 Sobel 结构提示	28

4.5	NAF 局部编码器	28
4.6	瓶颈处的共享/任务余弦原型混合	29
4.7	解码器与 skip connection	30
4.8	marker conditioning、FiLM 与 adapter	30
4.9	direct output 与零初始化 base/detail residual	31
4.10	Deep Supervision	31
4.11	当前实际模型规模	31
5	数据流向详解	32
5.1	用一个真实 patch 对照源码路径	32
5.2	训练数据流	34
5.3	验证数据流	35
5.4	测试与推理数据流	35
6	训练详解	36
6.1	epoch 与 batch: 把“看很多遍数据”说清楚	36
6.2	一次 batch 内发生什么	36
6.3	梯度累积与 effective batch	36
6.4	AMP 混合精度	37
6.5	AdamW、学习率、warmup 与梯度裁剪	37
6.6	EMA: 不是另一个独立模型	37
6.7	稳定 v2 对照的固定权重复合损失	37
6.8	v3/v4 两阶段损失、指标对齐与 D4 等变	38
6.9	channels-last、cuDNN benchmark 与 float32 matmul precision	39
6.10	进度条、异步指标、profiler 与 torch.compile	39
6.11	OOM 恢复	40
6.12	验证频率、checkpoint 与早停	40
7	验证详解	40
7.1	训练和验证的根本差异	40
7.2	SSIM 与 PSNR	41
7.3	float、uint8 与 JPG 三域	41
7.4	逐图、ROI、最差样本与分层聚合	41
7.5	bootstrap 的真实状态	42
7.6	raw、EMA 与 D4 的选择	42
8	测试与提交详解	42
8.1	为什么 test 不能显示本地分数	42
8.2	D4、权重来源与 JPEG 保存	43

插图	4
8.3 提交目录与命名	43
8.4 validate-submission 检查什么	43
9 配置系统：怎样读懂并安全修改 YAML	44
9.1 YAML 是什么	44
9.2 配置合并优先级	44
9.3 当前初赛配置逐项说明	45
9.4 GPU 加速选项与边界	46
9.5 当前通道数的一个重要事实	47
9.6 新手修改配置的安全原则	47
10 项目文件结构与模块职责	47
10.1 从根目录看项目	47
10.2 包入口与公共模块	48
10.3 数据模块	48
10.4 模型模块	49
10.5 损失与指标模块	51
10.6 训练、推理与实验引擎	51
10.7 提交与工具模块	52
10.8 outputs、artifacts 与 log 的区别	53
11 总结：从一张 DAPI 到竞赛 ZIP	53
A AutoDL 从上传到训练的保姆级流程	54
A.1 第 0 步：确认上传目录	54
A.2 第 1 步：激活环境并让 Python 找到项目	55
A.3 第 2 步：可选的 3 epoch 流程检查	55
A.4 第 3 步：v4 正式训练与验证	55
A.5 第 4 步：读取验证分数	56
A.6 第 5 步：对 test 推理并生成提交包	57
A.7 第 6 步：下载哪些文件	57
B 常见问题与排错顺序	58
C 实现事实核对清单	59

插图

1 医学显微图像的五个层级。左侧更接近独立生物/实验单位，右侧更接近神经网络直接处理的数值。	9
--	---

2	五类通道的典型亚细胞位置示意。marker 是分子探针，不是严格一一对应的细胞类别。	10
3	官方训练集同一位置的五个真实 marker 通道及 DAPI-CD68 教学伪彩叠加。	12
4	同一真实 patch 的全图与局部放大。DAPI 主要显示核，CD68 信号更多位于核周围和胞质区域。	13
5	autodl-run 的命令解析与主要调用链	19
6	按 CD68 全图平均强度选取的低、中、高活动度真实配对样本。这里的“活动度”是计算机统计量，不是临床阳性分级。	21
7	v4 继承的 MultiMarkerRestorer NAF 核心主干。图中按实际单逻辑通道标注输入/输出；当前重型三路增量见图 9。	25
8	历史 v3 增量路径，仅供兼容与对照；当前 v4 已用 Heavy Omni U 和三路门控替换轻量 U/固定频率组合。	25
9	v4 的三路主线。“并行”指三类特征在同一 autograd 计算图中分别计算，再由内容相关门控融合；不是三个进程各占一块 GPU。	27
10	NAFBlock：两次归一化、两条门控残差路径和零初始化残差缩放	29
11	MultiTaskPrototypeMixer：共享/任务原型注意力与残差融合	30
12	一个真实配对 patch 从 JPEG、Dataset、MultiMarkerRestorer 到损失与提交文件的代码路径。	32
13	训练时从 JPG 到模型参数更新的数据流	34
14	验证时的三域指标与 ROI 聚合数据流	35

表格

1	常见组织成像方式与本项目数据的区别	7
2	本项目五个通道的入门解释	11
3	数据样本中的字段与“标签”含义	14
4	不同评价层级的含义	16
6	本地官方数据的存储与逻辑通道事实	22
7	当前 manifest 划分结果	23
8	局部编码器各尺度	29
9	稳定对照、v3 与 v4 候选的结构规模	31
10	真实 patch 在代码中的表示变化	33
11	稳定 v2 对照的固定损失重建项权重	38
12	initial_round_cd68_max_v4.yaml 的关键生效配置	45
13	src/virtual_staining 顶层模块	48
14	src/virtual_staining/data 模块	48
15	src/virtual_staining/models 模块	49
16	损失和指标模块	51

17	src/virtual_staining/engine 模块	51
18	提交和工具模块	53
19	建议从 AutoDL 下载的产物	57
20	新手常见故障	58
21	截至本报告生成时的实现事实	59

1 引言与项目背景

1.1 什么是染色，为什么显微镜图像需要它

人体组织中的细胞通常接近透明。直接把一块薄薄的组织放到显微镜下，很多结构并不会自动呈现出清晰的颜色。实验人员会使用能够与特定分子结合的染料或抗体，把原本不明显的结构“点亮”。这件事可以类比为给城市地图加图例：道路、河流、医院本来都在地图上，但用不同颜色标出后，人才能快速分辨它们。

免疫组织化学染色（Immunohistochemistry, IHC）利用抗体识别组织中的目标蛋白。传统 IHC 往往需要制备切片、加入抗体试剂、孵育、清洗和显色，过程可能持续数小时，试剂和人工都有成本，而且一次实验会对样本产生不可逆处理。若希望观察多个标记物，通常还要进行更多实验。

1.2 H&E、IHC、免疫荧光不是同一种数据

初学者容易把所有“有颜色的病理图”都叫染色图，但它们的成像机制和计算含义不同。表 1 给出最小区分。本项目文件夹按 DAPI 与四个蛋白 marker 分通道保存，像素近似灰度强度；在缺少更完整的官方实验协议时，报告把它们作为**配准的荧光强度通道**处理，不自行猜测具体显微镜型号、荧光团或抗体克隆。

表 1: 常见组织成像方式与本项目数据的区别			
方式	实验上显示什么	计算机看到什么	与本项目的关系
H&E	苏木精偏向细胞核、伊红显示胞质和基质，是常规组织形态染色	通常是彩色 RGB 图，颜色由两种染料混合	本项目没有 H&E 输入，不能假设网络看到了其丰富组织颜色。
IHC	抗体识别目标蛋白，再用显色反应显示阳性区域	常见为明场彩色图，需处理染料颜色分离	CD68 等也常用于 IHC，但本项目磁盘图是独立 marker 强度通道，不应套用普通彩色 IHC 的颜色规则。
免疫荧光	抗体携带/连接荧光信号，不同分子由不同通道记录	每个通道本质是空间强度矩阵，可单独灰度显示，也可伪彩叠加	本项目的数据组织最接近这种“DAPI 加多个 marker 通道”的计算形式。
虚拟染色	不新增化学探针，由模型预测目标通道	输入张量经过神经网络得到连续强度图	预测图只是对实验图的统计估计，不能反过来当作真实做过一次染色。

1.3 虚拟染色是什么

虚拟染色 (Virtual Staining) 并不是让计算机真的把染料滴到玻片上，而是训练一个人工智能模型：给它一种已经拍摄好的图像，让它预测同一位置在另一种标记下应该呈现的像素强度。对本项目来说，可以把模型想象成一个非常复杂的“逐像素翻译器”：输入一张 DAPI 图，输出一张 CD68 图，输入和输出的尺寸、位置一一对应。

它的潜在价值包括：

- 减少重复染色所需的时间和试剂；
- 在不再次处理样本的前提下，探索另一种标记物的可能分布；
- 同一张 DAPI 图理论上可作为多个目标 marker 的共同输入；
- 预测是软件过程，可以批量运行并保留可复现的参数与日志。

需要强调的是，竞赛模型输出是**算法预测**，不是经过临床验证的真实检测结果。它可以用于竞赛、研究和方法开发，但不能因为图像“看起来像”就直接替代病理诊断。

1.4 DAPI 与 CD68 分别是什么

DAPI (4',6-diamidino-2-phenylindole, 4',6-二脒基-2-苯基吲哚) 是一种常用的蓝色荧光 DNA 染料。它与双链 DNA 的富含 A-T 区域结合后荧光明显增强，所以在荧光显微镜下常被用作细胞核复染。简单地说，DAPI 图像主要告诉我们“细胞核在哪里、密度如何、形状怎样”。DAPI 的常见性质可参考 Thermo Fisher 的技术说明 [8]。

CD68 (Cluster of Differentiation 68) 是一种主要位于内体/溶酶体系统的糖蛋白，在单核细胞和组织巨噬细胞中常见。病理研究中常用 CD68 抗体辅助观察巨噬细胞相关信号和炎症细胞浸润；Human Protein Atlas 也给出了相应的组织表达资料 [9]。但是 CD68 并非“只在巨噬细胞出现”的绝对身份证，一些非髓系细胞也可能表达，因此单凭 CD68 阳性不能完成细胞身份判定 [10]。为了入门，可以暂时把 CD68 图理解成“巨噬细胞相关信号的空间热力图”，同时牢记这只是近似说法。

只看 DAPI 并不能直接读取 CD68。模型需要从大量同位置的 DAPI-CD68 配对样本中学习统计规律：哪些核形态、密度和周围纹理经常与 CD68 信号共同出现。这个映射不是确定的化学公式，因此虚拟染色本质上是一个有不确定性的预测问题。

1.5 荧光显微镜图像与 patch

荧光显微镜图像 (Fluorescence Microscopy Image) 记录荧光分子在不同位置发出的光强。存储到文件后，每个像素是一个数字：数值较大通常代表该位置的信号较强。项目内部会把 8 位像素 0 ~ 255 转成浮点数 0 ~ 1，以便神经网络计算。

一张完整组织切片可能非常大，不能总是整张送入 GPU。因此数据制作者会把大图裁成许多固定大小的 **patch (图块)**。本地官方数据的每个 patch 是 256 × 256 像素。文件名如 ROI000_03_12.jpg，可拆成：

- ROI000: 第 0 个感兴趣区域;
- 03: 该 patch 在大区域网格中的行坐标;
- 12: 列坐标;
- .jpg: JPEG 文件格式。

ROI (Region of Interest, 感兴趣区域) 可以理解为从大切片中选出一块连续区域。同一 ROI 里的相邻 patch 可能原本就在大图上彼此相邻, 因此它们并不是完全独立的样本。

1.6 竞赛目标与图像到图像翻译

项目的核心任务是: 对每张测试 DAPI patch 生成同尺寸的 CD68 patch。模型学习的是

$$f_{\theta} : \mathbf{x}_{\text{DAPI}} \longrightarrow \hat{\mathbf{y}}_{\text{CD68}}, \quad (1)$$

其中 θ 是模型参数, $\mathbf{x}$ 是输入图, $\hat{\mathbf{y}}$ 是预测图。因为输入和输出都是图像, 而且像素位置需要对应, 所以它属于图像到图像翻译 (Image-to-Image Translation), 更具体地说也属于图像恢复/回归问题。

本地验证主要观察 SSIM (Structural Similarity, 结构相似性) 与 PSNR (Peak Signal-to-Noise Ratio, 峰值信噪比)。SSIM 更关注亮度、对比度和局部结构是否相似, PSNR 则由均方误差推导。两者通常都是越高越好, 但它们不能替代医学专家判断。

1.7 先建立空间层级: 组织、ROI、patch、细胞和像素

计算机视觉教材常从一张独立的图像讲起; 医学显微图像却有天然的层级结构。一张 JPG 并不等于一个独立患者, 也不一定等于一个独立生物样本。图 1 给出了本项目最重要的空间层级。

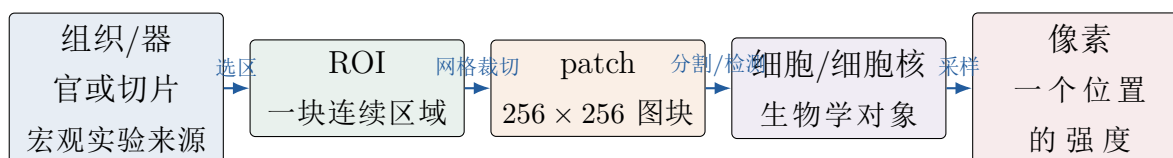


图 1: 医学显微图像的五個层级。左侧更接近独立生物/实验单位, 右侧更接近神经网络直接处理的数值。

- 组织/切片层: 实验者从某个生物样本得到组织切片。一张切片可包含大量细胞和多种组织结构。当前公开目录没有患者或玻片标识, manifest 的 organ 也为 unknown, 所以本报告不能把“一个 ROI”冒充“一个患者”。

- **ROI 层**: ROI 是切片中一块连续的感兴趣区域。不同 ROI 通常比同一 ROI 内的两个 patch 更接近统计独立，因此训练/验证划分必须优先按 ROI 分组。
- **patch 层**: 为了放入 GPU，大区域被切成 256×256 小图。文件名里的行、列坐标描述它在 ROI 网格中的位置；相邻 patch 可能共享相似的组织纹理和细胞群。
- **细胞层**: 病理学家关心的对象往往是细胞，例如“有多少 CD68 相关阳性细胞”“信号是否围绕某些细胞核出现”。要从图像得到细胞级量，通常还要先分割细胞核或细胞区域。
- **像素层**: 神经网络最终输出每个位置的亮度。像素是采样值，不是一个细胞，也不是一个分子。一枚细胞核通常覆盖许多像素，一个像素也可能混合多个亚细胞结构和背景光。

这解释了“像素级指标很好但细胞级结论可能错误”：模型可能把一个真实的亮点轻微平移或摊成一片较弱的光。大部分像素的误差仍然不大，但被判为阳性的细胞数、阳性细胞位置或局部表达强度已经发生变化。

1.8 一个细胞里有哪些结构，marker 为什么出现在不同位置

理解 marker 前，先把细胞极度简化为四个区域：**细胞核**储存 DNA；**细胞质**包含细胞器和骨架；**细胞膜**把细胞与外界分隔并承载许多受体；**细胞外基质**位于细胞之间并支撑组织。真实细胞没有规则圆形边界，而且二维切片只截取三维细胞的一层。图 2 只是空间位置的教学示意，不代表数据中的真实颜色、比例或细胞类型。

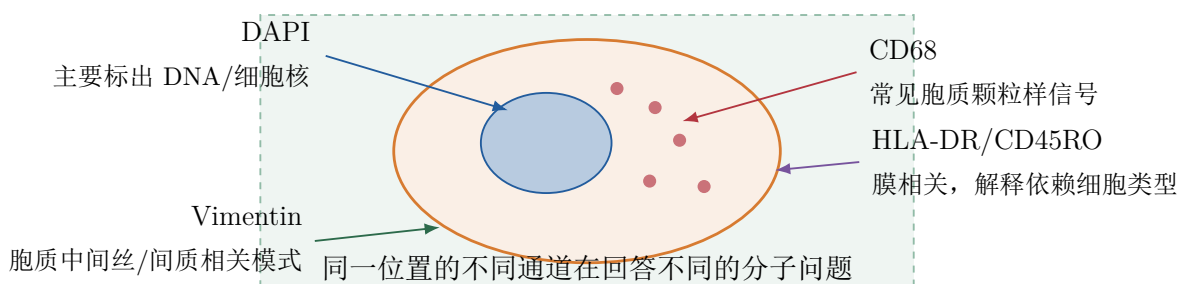


图 2: 五类通道的典型亚细胞位置示意。marker 是分子探针，不是严格一一对应的细胞类别。

1.9 本项目五个通道的医学含义

训练目录中同一个 basename 在五个 marker 文件夹各有一张图。它们可看作同一空间位置的五种“观察方式”。表 2 中的“常见关联”用于建立直觉，不应被当成临床诊断规则。

表 2: 本项目五个通道的入门解释

通道	主要分子/结构	常见医学关联	图像中可期待的空间形态	不能直接推出什么
DAPI	与双链 DNA 结合的核染料	提供细胞核位置、大小、形状和局部核密度	一个个核样亮区域；不同细胞的核形态可不同	看不见完整细胞膜；不能仅凭核图确定某蛋白一定表达
CD68	内体/溶酶体相关糖蛋白	常用于辅助观察单核细胞/巨噬细胞相关信号	常见胞质内、核周或颗粒样信号，而不是填满细胞核	不是巨噬细胞的绝对专属身份证；阳性不自动等于疾病诊断
CD45RO	CD45/PTPRC 的一种异构体	常与人类记忆/活化 T 细胞表型相关，但不同免疫亚群需联合其他 marker 判断 [11]	作为白细胞表面蛋白，常呈细胞轮廓/膜相关分布	不能只用该通道确定所有 T 细胞亚型、抗原特异性或功能状态
HLA-DR	人类 MHC II 类分子	抗原呈递细胞表面常见，也可在炎症刺激下于其他细胞上调 [12]	以细胞膜相关信号为主，也受胞内运输影响	不是单一细胞类型专属；强信号不能直接等同某种疾病
Vimentin	III 型中间丝细胞骨架蛋白	常见于间充质/基质相关细胞，且在多种细胞环境中表达	细胞质内纤维状、网状或弥散结构；不与细胞核完全重合 [13]	不能把所有 Vimentin 阳性区域都判成同一种细胞或固定生物过程

因此，从 DAPI 预测 CD68 等通道并不是简单地给细胞核“换颜色”。例如，一个 DAPI 核周围是否出现 CD68 颗粒，与细胞身份和状态有关；而 DAPI 本身没有直接测量该蛋白。网络只能利用训练集中反复出现的核形态、密度、组织纹理和蛋白信号之间的统计相关性。

1.10 把医学概念放回一张真实的五通道 patch

前面的示意图只说明“marker 可能出现在哪里”。图 3来自本地官方训练集的同个真实空间位置 R0I007_09_11，能更直观地说明五个通道并不是五张无关照片。前五格直接显示原始 uint8 灰度，没有对每个通道单独拉伸对比度；右下角只是教学用伪彩叠加，把 DAPI 映射为蓝色、CD68 映射为红色。数据文件本身并没有这些蓝红颜色。

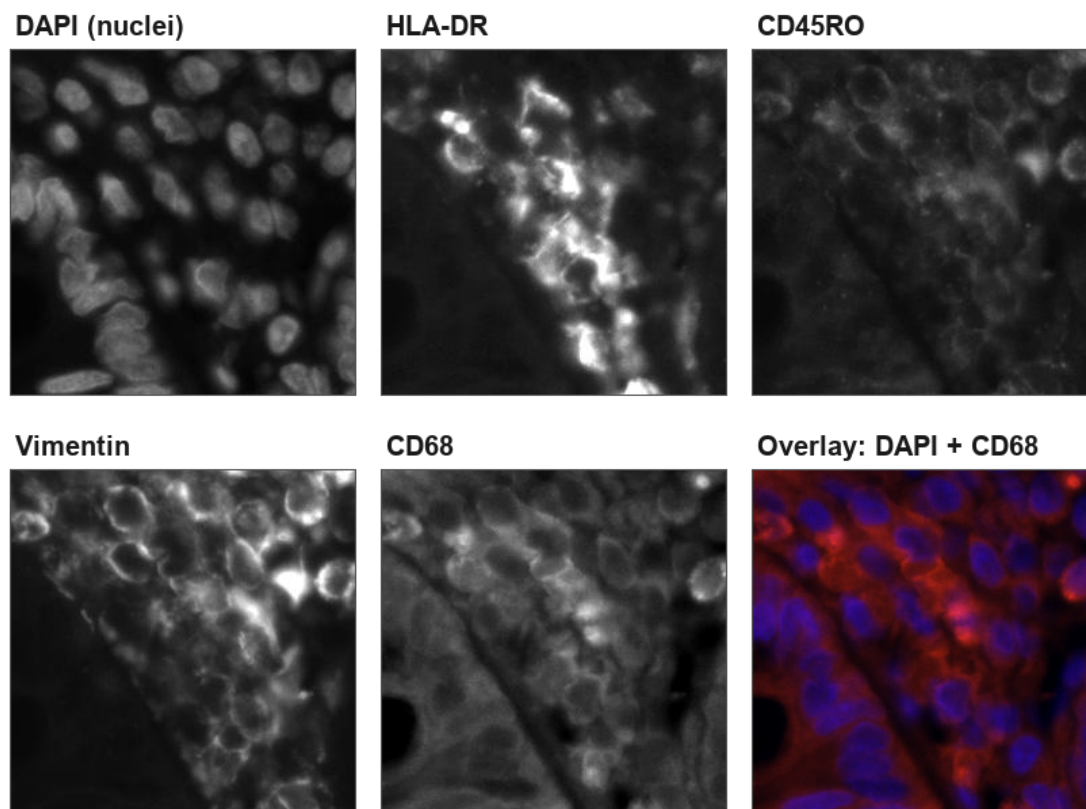


图 3: 官方训练集同一位置的五个真实 marker 通道及 DAPI-CD68 教学伪彩叠加。

读图时可以按下面的顺序观察：

1. **先看 DAPI：**一个个椭圆或不规则的亮区主要对应细胞核。它能提供核的位置和密度，但没有画出完整细胞边界。
2. **再横向比较 marker：**HLA-DR 在图中部有较集中的亮结构，CD45RO 的强度和纹理又不同；Vimentin 与 CD68 在不少位置呈胞质/轮廓相关信号。它们共享空间坐标，却不是 DAPI 的复制品。
3. **最后看叠加图：**蓝色核与红色 CD68 信号经常相邻、包围或部分重叠，而不是严格像素重合。这正是“核层级信息”和“胞质/蛋白层级信息”的区别。

这些观察只能描述本 patch 的图像形态，不能据此给每个细胞下临床诊断。例如，不能把所有红色区域直接数成巨噬细胞，也不能因为某个核周围没有明显红色就断言该细胞绝对不表达 CD68。严谨的细胞级结论还需细胞分割、多 marker 联合判定、实验阈值和病理专家复核。

图 4 进一步放大同一 patch 中 CD68 平均强度较高的 96×96 窗口。红框和裁剪位置由脚本根据 CD68 局部均值自动选择，并不是人工挑出的“典型阳性细胞”或医学标注。

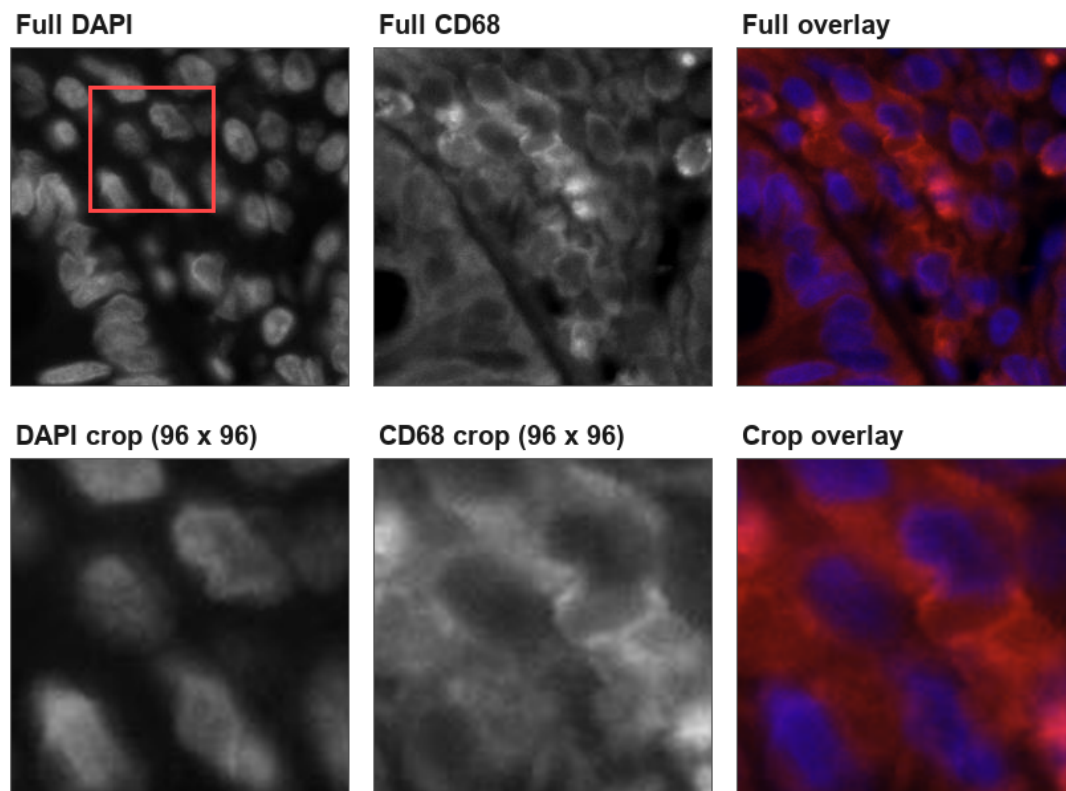


图 4: 同一真实 patch 的全图与局部放大。DAPI 主要显示核，CD68 信号更多位于核周围和胞质区域。

局部图也解释了为什么模型难：DAPI 给出蓝色核的形状，却没有直接告诉网络红色信号应当沿哪个核的哪一侧增强。网络只能从训练样本中学习常见条件关系；当 DAPI 外观相似而 CD68 状态不同时，像素回归会倾向输出较平滑的平均结果。

1.11 本项目的 label 到底是什么

计算机课程里，label 常指“猫/狗”或 0/1。本任务不同：主要监督信号是一整张目标 marker 图，称为 **稠密标签 (dense target)**。每个目标像素都是一个回归值，而不是给整个 patch 一个类别。

表 3: 数据样本中的字段与“标签”含义

信息	计算机表示	含义与用途
DAPI 输入	$H \times W \times C$ 图像	模型可见的条件信息；当前 JPG 存储为 RGB，但三通道相同。
目标图	同尺寸连续强度图	train 中 HLA-DR、CD45RO、Vimentin、CD68 都存在；当前初赛配置只把 CD68 作为训练目标。
marker 名	字符串/任务 ID	告诉多任务模型要预测哪种蛋白通道；它不是患者诊断。
ROI、row、col	分组与整数坐标	用于防止泄漏、恢复邻域和按区域汇总；不直接参与当前默认 context-disabled 模型。
split	train/val/test	train 用于更新权重，val 用于选模型，test 只有 DAPI 且隐藏目标由竞赛平台保管。
organ	元数据字符串	当前本地 manifest 为 unknown ；在没有官方元数据时不能从文件夹结构臆造器官标签。

本地 official 数据的结构是：训练集中每个 marker 各 6296 张、可组成 6296 组同名配对；按真实 ROI 分组后得到 5004 个 train patch 和 1292 个 val patch。测试目录有 1346 张 DAPI，但没有任何目标图。所以 val 分数是本地可计算的开发指标，test 分数必须把预测提交到平台后才能得到。

还要区分“图像强度”和“蛋白分子数”。荧光图中的亮度受染色效率、曝光、探测器、背景、自发荧光、漂白和饱和等因素影响；若没有逐批校准，像素值通常只是相对强度，不等于一个可以跨实验直接比较的绝对分子计数 [14]。因此网络拟合的是数据制作流程下的目标图像分布，而不是把 0 ~ 255 机械地翻译成“多少个蛋白分子”。

1.12 配对与配准：同名文件为什么重要

监督学习需要知道哪一张 DAPI 对应哪一张目标图。本工程用 marker 文件夹和规范化后的同名 stem 建立配对，例如：

```
train/DAPI/ROI000_03_12.jpg
train/CD68/ROI000_03_12.jpg
```

相同的 ROI000_03_12 表示二者应来自同一网格位置；这叫**空间配对**（spatial pairing）。但“文件名相同”不等于所有边缘都精确重合：制样、通道采集、光学系统和压缩仍可能造成轻微位移或噪声。本地审计得到目标通道与 DAPI 的中位平移为 0 像素，说明没有发现系统性整体错位，所以默认不做配准；这不代表每个细胞的所有亚细胞信号都应该与 DAPI 核重合。CD68 本来就主要不是核内信号。

若训练/验证把同一 ROI 的相邻 patch 随机拆开，模型可能在验证时看见几乎相同的组织环境，导致分数虚高。这叫**数据泄漏 (data leakage)**。本项目按 ROI 分组，当前 train 与 val 的 ROI 不重叠。对于未来含患者或玻片 ID 的数据，应该优先按患者/玻片分组，因为它们比 ROI 更高一层。

1.13 从光子到 Tensor：计算机实际接收了什么

荧光显微镜把某个波段的光转换成探测器读数，保存前又可能经过曝光、增益、位深转换和压缩。进入当前工程后，一张图经历下面的表示变化：

1. 磁盘上的 JPEG 解码为 $256 \times 256 \times 3$ 的 uint8 数组，元素为整数 $0 \sim 255$ ；
2. 图像库按 RGB 顺序读取。审计发现三通道逐像素相同，所以医学信息上是一个灰度强度通道；
3. 数据集把像素除以 255，变成 float32 的 $0 \sim 1$ ；
4. 维度从 HWC 调整为 PyTorch 的 CHW，再加 batch 维成为 $B \times C \times H \times W$ ；
5. 网络输出同范围的浮点预测；提交时先四舍五入量化回 uint8，再以固定 JPEG 参数编码。

JPEG 是有损格式。同一个浮点预测在“内存 float”“量化 uint8”“写成 JPG 后重新读取”三个域里并不完全一致，这也是项目同时报告三域指标的原因。背景、自发荧光和探测器噪声会减少有效动态范围，饱和像素则丢失超过上限的强度信息；这些都是定量荧光显微分析的常见限制 [14]。

1.14 模型学的是条件预测，不是把不可见信息恢复出来

更准确的数学说法是，训练数据从某个联合分布产生 $(X_{\text{DAPI}}, Y_{\text{marker}})$ ，模型试图学习

$$\hat{Y} = f_{\theta}(X) \approx \mathbb{E}[Y | X]. \quad (2)$$

如果两个细胞的 DAPI 外观几乎相同，但一个表达 CD68、另一个不表达，那么仅凭 DAPI 无法唯一判别。在均方误差等损失下，模型容易给出两种可能性的平均，即一个偏弱、偏平滑的信号。这不是简单增加网络参数就一定能解决的问题，而是**条件不确定性 (conditional uncertainty)**：输入中可能缺少决定目标的分子信息。

因此应避免三种过度表述：其一，“模型检测到了 CD68 蛋白”；准确说法是“模型预测了 CD68 图像”；其二，“亮点就是巨噬细胞”；准确说法是“亮点与 CD68 相关，细胞身份仍需联合形态和其他 marker”；其三，“SSIM 提高就有临床价值”；临床价值还需要外部队列、细胞级分析和专家验证。

1.15 像素级、细胞级、patch 级与 ROI 级评价

表 4 把后文经常出现的“层级”拆开。一个指标属于哪一层，决定了它能回答什么问题。

表 4: 不同评价层级的含义

层级	计算对象	可以回答的问题	局限与常见指标
像素级	每个坐标的预测强度与真值强度	整体灰度和局部结构有多接近？	MSE、PSNR、SSIM；不理解细胞身份，且大量暗背景会稀释稀有亮点的错误。
细胞级	先用 DAPI 分割核/细胞，再对每个对象汇总 marker	阳性细胞比例、单细胞平均/最大强度、信号与核的距离是否合理？	依赖分割和阳性阈值；本项目当前标准 Validator 尚未计算。
patch 级	一张 256×256 图的指标或统计量	哪些局部图块最难？性能是否随亮度、密度变化？	同一 ROI 内 patch 高度相关，不能把它们全当独立样本。
ROI 级	同一 ROI 的全部 patch	一个连续组织区域的平均表现和空间一致性如何？	ROI 数通常远小于 patch 数；本项目用 ROI 聚合和成对 bootstrap 做可靠比较。
切片/患者级	同一玻片或患者的多个 ROI	方法能否跨患者泛化，是否改变患者级结论？	当前数据没有可验证的患者/玻片 ID，不能声称通过患者级验证。

一个简单反例能说明为什么要分层：真值里有 2 个强阳性细胞，预测把相同的总亮度均匀分给 10 个细胞。两张图的总强度可能相近，某些像素平均误差也未必极端，但“阳性细胞数量”和“信号集中在哪些细胞”已经完全不同。反过来，如果预测只比真值平移 1 个像素，细胞级阳性结论可能不变，像素级 MSE 却会明显增大。

1.16 SSIM、PSNR 与医学意义应该怎样对应

给真值图 Y 和预测图 $\hat{Y}$ ，均方误差为

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2, \quad (3)$$

当图像范围归一化为 $[0, 1]$ 时，PSNR 为

$$\text{PSNR} = 10 \log_{10} \left(\frac{1}{\text{MSE}} \right). \quad (4)$$

PSNR 高表示平均平方误差小，但它对整体亮度范围敏感，也不知道一个误差落在背景还是关键阳性细胞上。SSIM 在局部窗口比较亮度、对比度和结构 [4]，比单纯 MSE 更接近“纹理是否相似”，但它也没有细胞语义：两团形状相似却属于不同细胞的信号，SSIM 不会自动理解其医学差别。

本报告后文的 float/uint8/JPG 指标仍属于同一套像素/局部结构指标在三种文件表示上的计算，不是三个医学终点。若要进一步研究医学一致性，可在不改变竞赛主指标的前提下增加：单细胞 marker 强度相关、阳性细胞比例误差、细胞间强度排序的 Spearman 相关、ROI 内空间分布差异等。它们必须固定细胞分割方法和阈值，并报告失败样本，不能挑一张好看的热力图代替统计检验。

1.17 读后续网络章节前应掌握的最小知识清单

1. 一张 target 图是稠密回归标签，不是一个类别编号；
2. DAPI 主要描述细胞核，CD68/HLA-DR/CD45RO/Vimentin 描述的是不同分子和亚细胞位置；
3. marker 与细胞类型是“相关但不绝对唯一”的关系，细胞身份通常需要多 marker 与形态联合判断；
4. 一个 ROI 含很多相邻 patch，随机按 patch 划分会造成泄漏；
5. 像素、细胞、patch、ROI、患者是不同统计单位，不能混用样本量；
6. 荧光亮度是受实验流程影响的相对测量，不应直接称为蛋白分子数；
7. 网络预测条件分布中的常见结果，不能凭 DAPI 创造输入中完全不存在的确定信息；
8. PSNR/SSIM 衡量图像相似度，不自动证明细胞级或临床级正确；
9. test 没有公开目标，验证命令只能评估 val，平台分数才是隐藏 test 的结果；
10. JPEG 量化/压缩会改变像素，所以最终竞赛比较应重点看 JPG round-trip 指标和平台结果。

2 命令行入门：为什么用 CLI 而不是 python train.py

2.1 CLI 是什么

CLI (Command Line Interface, 命令行接口) 是通过输入文字命令控制程序的方式。图形界面像餐厅菜单，CLI 更像把“菜名、份量、备注”写成一行交给厨房。它看起

来不如按钮直观，但非常适合服务器：命令可以复制、记录、重跑，也能清楚说明使用了哪个配置和数据目录。

本项目推荐的训练命令是：

```
python -m virtual_staining.cli --log-root log autodl-run \
  --config configs/initial_round_cd68_max_v4.yaml \
  --data-root AUTO --target CD68 --run-id cd68_max_v4_seed2026
```

2.2 把命令逐段拆开

命令片段	含义
python	启动当前环境中的 Python 解释器。AutoDL 上应先激活装好 PyTorch 的环境。
-m	告诉 Python：“后面给的是可导入模块名，请按模块运行”，而不是磁盘上的单个脚本路径。
virtual_staining.cli	Python 模块路径，对应源码 src/virtual_staining/cli.py。安装项目后，Python 能从 src/ 布局中找到它。
--log-root log	全局选项。每次 CLI 调用会在根目录 log/ 下创建轻量日志目录，记录环境、命令、配置、epoch 指标和错误；成功结束后还会打成日志 ZIP。该参数必须放在子命令之前。
autodl-run	子命令，表示执行“环境检查、数据准备、训练和验证”的一键编排。
--config ...yaml	指定本次使用的 YAML 配置。YAML 就像一张集中填写超参数的表。
--data-root AUTO	不硬编码目录，让发现程序在受限候选路径中评分并选择数据根；当前应发现 dataset/official。
--target CD68	通用训练/验证命令可用的目标参数。需要注意：当前 autodl-run 包装器实际从 YAML 的 data.targets 和 data.submit_targets 读取唯一目标；本配置本来就是 CD68，因此该参数起到说明意图的作用，但不要误以为它会覆盖包装器中的 YAML 目标。
--run-id ...	本次实验的唯一名字，同时用于输出目录和日志目录。已有非空输出目录不会被静默覆盖。

2.3 为什么不是直接运行 train.py

项目根目录并不存在一个承担全部工作的 `train.py`。代码采用标准 `src 布局 (src layout)`：可导入包位于 `src/virtual_staining/`。使用 `-m` 有三点好处：

1. Python 按包规则解析模块间导入，不依赖你碰巧站在哪个子目录；
2. 所有功能共享同一个入口，却可通过子命令区分 `train`、`validate`、`predict`、`autodl-submit` 等流程；
3. 参数解析、日志、异常退出码和配置合并方式保持一致，便于服务器自动化。

如果已经执行 `pip install -e .`，包会以 `editable`（可编辑）方式注册到当前 Python 环境；修改 `src/` 后不需要反复复制包。若不安装，也可以把根目录下的 `src` 加进 `PYTHONPATH`。

2.4 从入口到训练器：autodl-run 内部流程

图 5 展示了命令的路由关系。

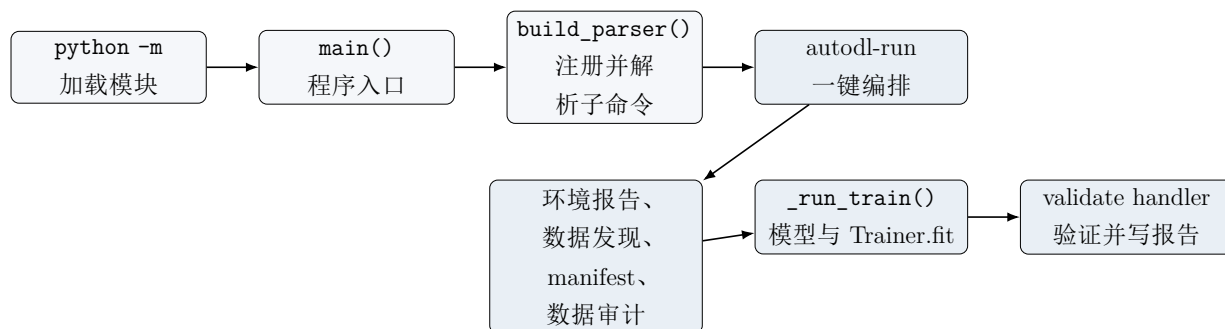


图 5: autodl-run 的命令解析与主要调用链

真实代码的执行顺序如下：

1. 模块末尾调用 `main()`；`build_parser()` 创建 `argparse` 解析器并注册所有子命令。
2. 参数解析完成后，`ExperimentLogSession.from_args()` 先在 `log/<run-id>/` 创建 `invocation`、`environment` 和 `status` 文件，并把控制台输出同时写入日志。
3. 解析器发现子命令是 `autodl-run`，调用 `command_autodl_run()`。
4. `command_env()` 收集 Python、PyTorch、CUDA、GPU、CPU 和内存信息，写入 `artifacts/environment.json`。
5. `discover_data_root()` 对有限候选目录打分；当前数据根为 `dataset/official`。
6. `_ensure_manifests()` 配对文件并建立 `train`、`val`、`test` 与 `smoke` CSV；随后 `audit_data()` 统计尺寸、通道、值域、相关性和对齐情况。

7. `_run_train()` 合并配置、构建带 `hard-mix activity sampler` 的 `DataLoader`、`MultiMarkerRestorer`、`scheduled composite loss`、分组 `fused AdamW`、`Trainer` 与 `Validator`；`Trainer.fit()` 最多训练 100 epoch，并受 `early stopping` 与 8.5 小时预测式安全停点约束。
8. v4 每 5 epoch 比较 raw 与 EMA；15% batch 还会执行一次随机 D4 等变一致性分支。训练结束后，`command_validate()` 加载最佳 checkpoint 中记录的选中权重来源，输出逐图和汇总指标；当前 v4 推理固定使用 `tta=d4` 与 `tta\_policy=configured`。
9. `pipeline_report.json` 汇总环境、数据、训练和验证路径。CLI 成功结束时，日志会复制关键轻量文件并生成 `autodl-run_log_bundle.zip`。

2.5 autodl-submit 内部流程

提交命令不会再训练。它依次做以下事情：

1. 若给了 `--checkpoint` 就使用它；否则在 `outputs/**/best_ssim.ckpt` 中选修改时间最新的文件。
2. 重新发现数据并建立 test manifest，确认 test 不是空表。
3. `command_predict()` 根据 checkpoint 内保存的模型架构和 `ImageSpec` 构建模型，选择 raw 或 EMA 权重，按需要执行 D4 TTA，并把预测保存到 `predictions/CD68/`。
4. `command_make_submission()` 将预测复制为规定命名，构建 `results/test/CD68/` 并压缩为 ZIP。已经是 JPEG 的预测会直接复制，避免二次有损编码。
5. `validate_submission()` 检查文件名、数量、尺寸、模式、像素范围、纯黑/纯白异常、ZIP 层级和 CRC。任一硬错误都会令命令返回非零退出码。

3 数据集格式与加载

3.1 本地官方数据的真实目录与数量

当前工作区的真实目录为：

```
dataset/official/
|-- train/
|   |-- DAPI/          6296 JPG
|   |-- HLA-DR/        6296 JPG
|   |-- CD45RO/        6296 JPG
|   |-- Vimentin/      6296 JPG
|   `-- CD68/          6296 JPG
```

```

`-- test/
  `-- DAPI/      1346 JPG

```

训练目录每个 marker 的文件名一一对应，总计 6296 组配对样本。当前初赛配置只选择 CD68，因此每次训练实际读取 DAPI 与 CD68；其余三个目标仍在磁盘和 manifest 中，但不参与本次单目标 loss。test 只有 1346 张 DAPI，没有 CD68 标签，所以本地无法为 test 计算 SSIM 或 PSNR。

3.2 真实样本显示了哪些数据分布特征

仅展示一张“好看的图”容易造成误解。图 6 先对 6296 张训练 CD68 图计算归一化平均强度，再选取接近第 10、50、90 百分位的三个样本。每行左侧是模型输入 DAPI，中间是训练目标 CD68，右侧是教学伪彩叠加；mean 是 CD68 全图平均强度， >0.25 表示强度超过 0.25 的像素比例。

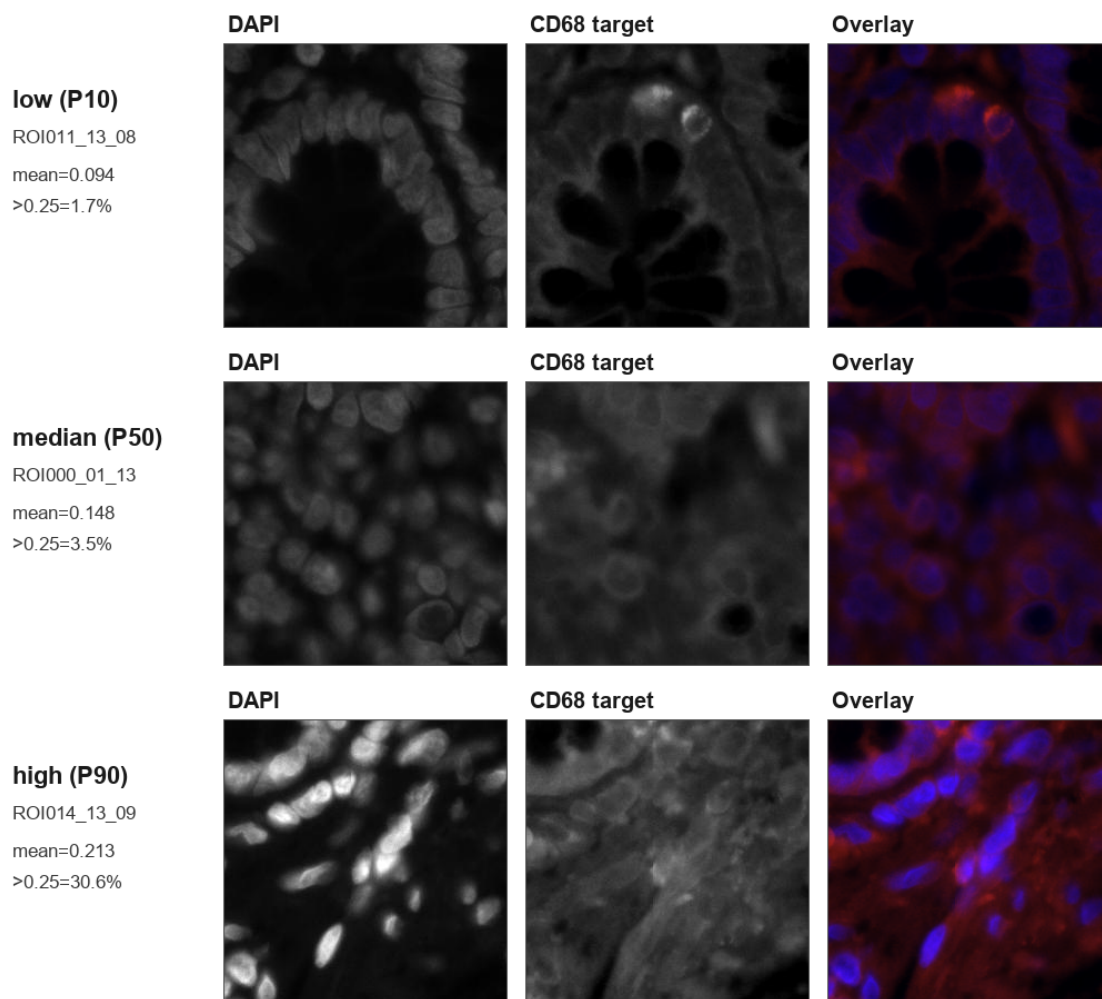


图 6: 按 CD68 全图平均强度选取的低、中、高活动度真实配对样本。这里的“活动度”是计算机统计量，不是临床阳性分级。

这三行揭示了四个与建模直接相关的特征：

- **目标强度分布不均匀**：不同 patch 的背景、亮区面积和局部峰值相差明显。若训练只优化全图平均误差，大量暗背景可能压过少量但重要的亮结构。
- **DAPI 与 CD68 不是确定映射**：核密度、核亮度与 CD68 有统计联系，但从三行不能得到“DAPI 越亮，CD68 必然越亮”的单调规则。
- **标签是连续图而非二分类 mask**：CD68 目标包含从暗背景到强信号的连续值。报告中的 0.25 只用于展示像素占比，不是项目训练标签，也没有被当作临床阳性阈值。
- **结构与强度都要学习**：网络既要预测哪里出现信号，又要估计信号有多强；这也是复合损失同时使用 MSE/Charbonnier 与 SSIM、梯度、金字塔项的动机。

选择过程可由 docs/essay/build_report_figures.py 完整复现，具体样本、统计量和显示规则记录在 docs/essay/figures/report_figure_provenance.json。脚本只读取原始数据，不会覆盖、重编码或移动 dataset/official 中的任何文件。

3.3 “RGB 文件”和“灰度内容”不是同一概念

本地 artifacts/data_audit.json 的全量审计结果为：所有 DAPI 和四个目标图都是 256×256 、8 位 JPEG、PIL mode RGB、3 个存储通道；同时，三个通道之间的平均差和最大差均为 0。因此，图像内容在逻辑上是灰度，文件容器却是 RGB。

表 6: 本地官方数据的存储与逻辑通道事实

Marker	图数	尺寸	存储通道	逻辑通道	PIL mode
DAPI	6296	256×256	3	1	RGB
HLA-DR	6296	256×256	3	1	RGB
CD45RO	6296	256×256	3	1	RGB
Vimentin	6296	256×256	3	1	RGB
CD68	6296	256×256	3	1	RGB

initial_round_cd68_max_v4.yaml 明确写入 1 个逻辑输入/输出通道，配置加载器还会合并 configs/resolved_local.yaml；该文件根据审计结果设置 data.input_channels: 1，并把四个 target 的逻辑通道数都设为 1。因此当前 Dataset 与模型实际按 $[B, 1, 256, 256]$ 运行，输出也是单通道逻辑张量。这里的“单通道训练”不等于把原始文件改成 mode L：官方 JPEG 仍保持只读 RGB，提交保存时仍按 checkpoint 中记录的 RGB storage mode 恢复三个相同通道。

3.4 文件名、ROI 与二维坐标

文件名使用 ROI{编号}_{行}_{列}.jpg。manifest 中旧的 parse_roi_id() 会提取 ROI000 用于分组；更严格的 parse_patch_coordinate() 可进一步解析行列，供网格审

计、边界/内部分类和邻域模型使用。当前 v4 配置的 `model.context.enabled=true`，但 `require_verified_grid=true`：只有文件名坐标、行列方向、边界连续性和 split 隔离审计全部通过，才会读取同一 ROI 的 3×3 DAPI 邻域；缺失位置由 mask 表示，目标 marker 永远不作为 context 输入。

坐标有空洞并不一定代表数据损坏。例如 ROI 网格可能只保留组织区域，背景 patch 被排除。程序只把真实存在的文件写入 manifest，不会凭空生成缺失坐标。

3.5 manifest 是什么

manifest（清单文件）是 CSV 表格。它不存像素，只存“到哪里找图”和“这张图属于谁”。每一行包含 organ、split、ROI、patch ID、规范化 key、DAPI/目标相对路径、宽高、通道、mode、文件大小、SHA-1 指纹、是否配对及缺失标记等。这样，训练过程可以被审计：某个预测出错时，能追溯到原始文件。

路径写成相对于数据根的形式，例如 `train/DAPI/ROI000_00_00.jpg`，运行时再与数据根拼接。代码使用 `pathlib.Path`，因此支持中文和空格路径。

3.6 按 ROI 划分训练集与验证集

官方目录没有单独的 val 文件夹。程序先从文件名得到 25 个训练 ROI（ROI000–ROI024），再以种子 2026 按组抽取约 20% ROI 作为本地 val。当前清单结果见表 7。

表 7: 当前 manifest 划分结果

集合	patch 数	ROI 数	说明
train	5004	20	用于反向传播与参数更新
val	1292	5	ROI004、ROI005、ROI006、ROI011、ROI023；不参与参数更新
test	1346	5	只有 DAPI；用于生成提交

当前泄漏报告显示 train 与 val 的 canonical key、DAPI 哈希和 group ID 交集均为空，ROI 来源为可靠的文件名正则，因而这次 ROI 分组划分是可验证的。为什么不能简单随机抽 20% patch？因为同一 ROI 的相邻 patch 很相似。如果左边 patch 进训练集、右边 patch 进验证集，模型可能靠记忆局部区域获得虚高分数。按 ROI 分组相当于把整本练习册分开，而不是把同一道题的上下半部分分别放到训练和考试中。

3.7 Dataset 与 DataLoader 如何把 JPG 送入模型

`VirtualStainingDataset.__getitem__()` 的核心步骤是：

1. 从 manifest 取一行，拼出 DAPI 和 CD68 的绝对路径；

2. PIL 解码 JPEG。灰度存储会转 L，其他存储会转 RGB；当前文件因此得到 HWC RGB 数组；
3. 因逻辑通道数为 1，对三个相同 RGB 通道取均值并四舍五入，再转为 CHW 张量、除以 255，得到 float32 的 [0,1] 值；
4. 检查 DAPI 与目标空间尺寸完全相同；
5. 训练时调用成对增强，最后再次转 float32 并截断到 [0,1]；
6. 返回输入、targets 字典、stem、ROI、organ、split 和路径元数据。

v4 YAML 在 AutoDL/Linux 上使用 `batch_size=6`、`gradient_accumulation=2`、`num_workers=8`、`prefetch_factor=4`、`pin_memory=true` 与 `persistent_workers=true`，以并发解码减少 4090 等待。加载器只在 CUDA 可用时真正 pin memory；Windows 本地若 worker 不稳定，可用 CLI 覆盖为 0，此时程序也会关闭 persistent。每次运行都应以日志中的 effective config 为准，而不能只看源 YAML 猜测最终生效值。

3.8 PairedTransform 为什么必须“成对”

若只翻转 DAPI、不翻转 CD68，标签就错位了。因此水平翻转、垂直翻转、0/90/180/270 度旋转和可选平移会对输入与所有目标同步执行。亮度、对比度、gamma、噪声和模糊则只施加在 DAPI 输入上，用来模拟采集强度变化，同时保留真实 CD68 标签。当前通过 `_make_loader()` 创建普通 PairedTransform 时，只从配置传入 `max_translation`；其余参数使用类的默认值：水平/垂直翻转概率 0.5、旋转概率 1.0、gamma/亮度/对比度概率各 0.5、噪声概率 0.25、模糊概率 0，平移默认为 0。

4 模型架构详解

4.1 先建立整体直觉：这是一个“先压缩理解，再放大还原”的网络

当前 v4 候选以 MultiMarkerRestorer 为核心，可以看成带三条互补特征路线的 U-Net 风格编码器-解码器 [1]。编码器逐级减小宽高、增加通道，把像素转成更抽象的特征；解码器再逐级放大，并从编码器拿回高分辨率 skip feature，恢复细节。与普通 U-Net 不同的是，本模型用 NAF block 负责恢复，在 1/8 瓶颈用共享/任务余弦原型混合语义，并在每个解码尺度用 CD68 task adapter 做条件调制。图 7 先画出 v4 从稳定 v2 继承的 NAF 主干；图 9 再展开当前真正启用的 Heavy Omni U、自适应频率、内容感知融合与 ROI context。v3 轻量分支只作为历史对照单独保留，绝不是当前 v4 前向的一部分。

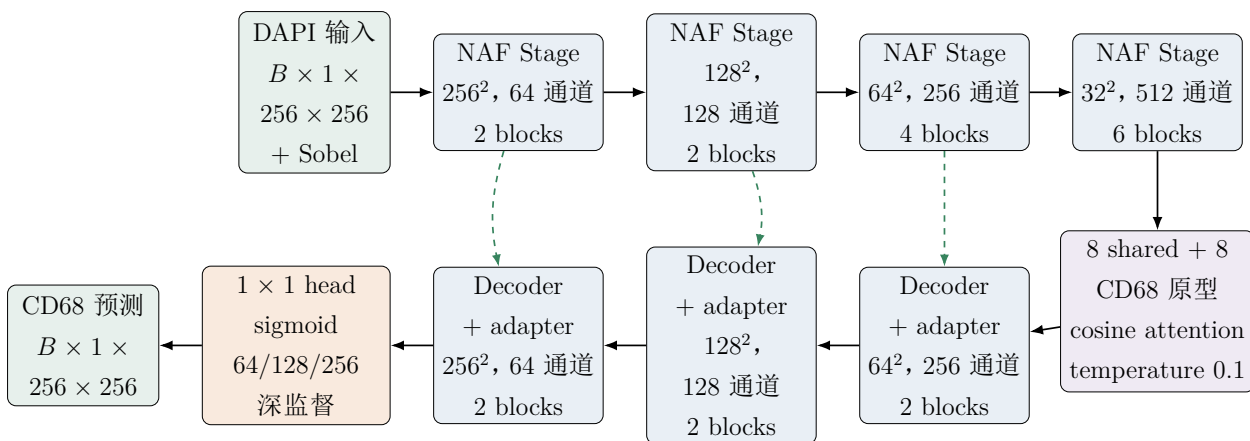


图 7: v4 继承的 MultiMarkerRestorer NAF 核心主干。图中按实际单逻辑通道标注输入/输出；当前重型三路增量见图 9。

4.2 历史 v3 对照：轻量 U、固定频率与稳定输出（当前未启用）

本小节用于解释旧配置和旧 checkpoint，不能拿来描述当前 v4。v3 候选不是把主干替换成一个更大的 Transformer，而是在已验证的 NAF U 形网络旁边增加四条互补、可由 feature flag 独立关闭的路径，如图 8。这遵循高效卷积复原网络的经验：局部大核/深度可分离卷积可以用较少参数补充细节 [15, 18]；频率分解可以显式区分平滑强度与高频纹理 [19]；训练期等变约束可以减少旋转/翻转后的输出不一致 [20]。这些文献只提供设计依据，本工程是针对 DAPI→CD68 和单卡 4090 预算的 clean-room 实现，不复制论文权重或结果。

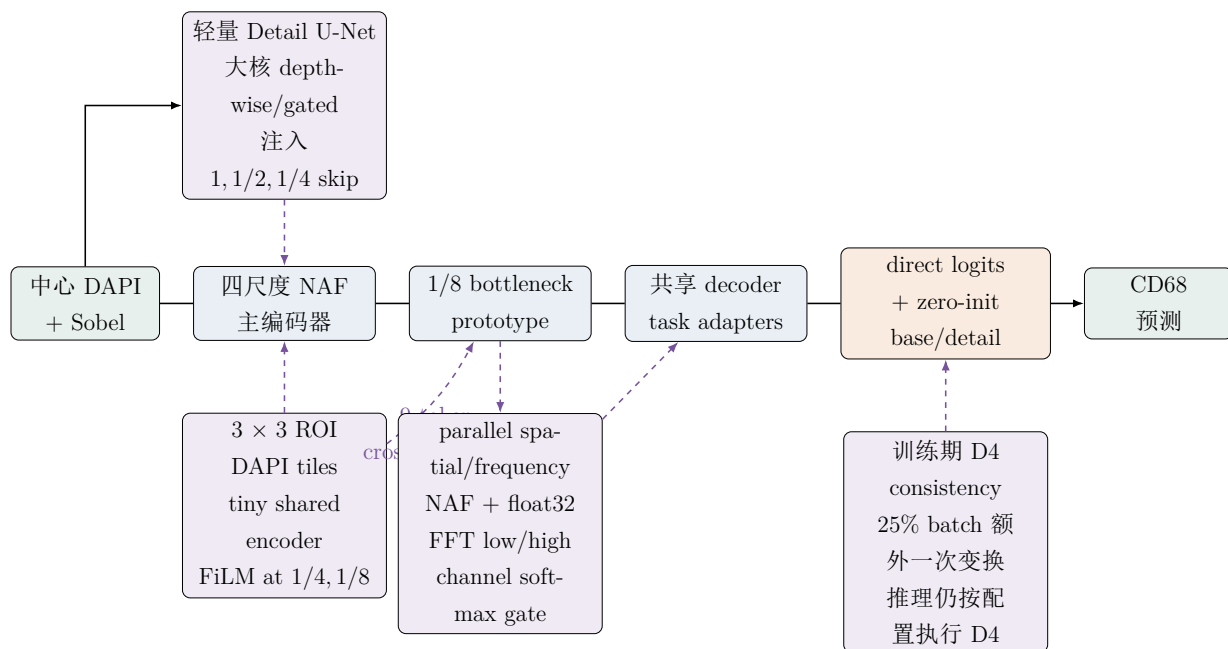


图 8: 历史 v3 增量路径，仅供兼容与对照；当前 v4 已用 Heavy Omni U 和三路门控替换轻量 U/固定频率组合。

轻量 Detail U-Net。 它使用宽度 [16, 24, 32, 48] 的四尺度 U 形分支，每层由 7×7 depth-wise convolution、 1×1 expansion 和乘法 gate 组成。解码结果只向主干的全分辨率、1/2、1/4 skip 注入；1/8 留给空间-频率 mixer，避免重复计算。三个注入投影严格零初始化，所以启用分支时第 0 步不改变旧主干输出。该分支约增加 72,016 个参数和 0.589 GMAC。

空间-频率瓶颈。 同一个 32×32 bottleneck 一路进入局部 NAF，一路在 float32 中执行 rFFT，使用平滑径向 mask 拆成 low/high bands，再由逐通道 softmax gate 融合。最终 projection 乘零初始化 residual scale 后接回主干。FFT 强制 float32 是为了 CUDA 数值/shape 兼容，其余卷积继续在 BF16 autocast 下运行。

真实 ROI context。 文件名 ROI007_09_11 可解析为 ROI、row、col；数据审计已经确认官方 train/val 没有共享 ROI 或跨 split 邻接边，行列方向连续。NeighborhoodDataset 因此可以安全读取中心周围最多 9 张 DAPI，并返回 valid mask 与二维 offsets。增强会同步变换 tiles、mask 和 offsets；中心 tile 强制与主输入逐位一致。若换数据后文件名网格无法验证，require_verified_grid=true 会拒绝训练，不会根据图像内容猜坐标。

稳定 base/detail。 旧 base/detail 会用随机新头直接替换最终 output head，单次重训风险较高。v3 引入且 v4 继续使用的 base_detail_residual=true 改为

$$z_{\text{final}} = z_{\text{direct}} + \Delta z_{\text{coarse}} + a \tanh(\Delta z_{\text{detail}}), \quad a = 0.5, \quad (5)$$

其中 coarse 和 detail 的最终投影都从 0 初始化。因此第 0 步 prediction 与旧 direct head 逐位一致，但两个投影在第一次 backward 都能获得梯度。

4.3 v4 重型三路主线：为什么替换轻量 U，却不删除旧模型

v3 的轻量 Detail U 只有 72,016 个参数，只向三个 skip 输出零初始化更新，没有独立的解码细化和 1/8 瓶颈路线。它适合做低风险增量，但不能满足“重型 U-Net 与另外两路平行后再融合”的设计意图。因此 v4 使用新的 models/heavy_detail_unet.py，宽度为 [56, 88, 128, 176]，编码深度为 [3, 4, 6, 6]，解码细化深度为 [1, 2, 3]。全分辨率和 1/2 尺度使用 3×3 depthwise spatial mixing 与稠密 1×1 gated channel mixer，便于 4090 Tensor Core 批量执行；1/4 和 1/8 尺度再并行混合局部 3×3 、 $1 \times 11 + 11 \times 1$ 轴向大核与空洞卷积。这一 clean-room 组合受 ConvIR 的多尺度卷积恢复、OKNet 的 omni-kernel 及 DarkIR 的多尺度空间建模启发 [15, 18, 16]，但没有复制它们的预训练权重或实验数字。

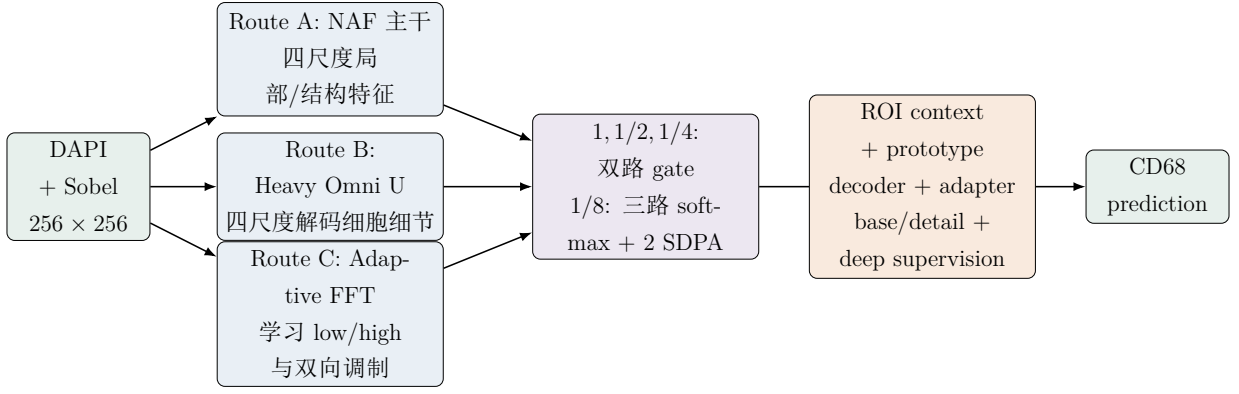


图 9: v4 的三路主线。“并行”指三类特征在同一 autograd 计算图中分别计算，再由内容相关门控融合；不是三个进程各占一块 GPU。

自适应频率路线。 v3 对每张图都使用固定 cutoff 0.35。v4 在这个稳定先验上根据当前 patch 预测有界的 cutoff 偏移和 transition 宽度，然后让池化的 high band 调制 low band, low band 反过来调制 high band。这一“先分频、再双向交互”的思路受 AdaIR [19] 启发，但要特别注意：AdaIR 处理噪声、雨、雾等退化，本项目是 DAPI→CD68 跨 marker 映射，所以这只是候选归纳偏置，不能由论文结果推导一定涨分。FFT 仍固定使用 float32，学习投影仍享受 BF16 AMP。v4 调用 `ParallelSpatialFrequencyMixer.forward_route()` 取得不含输入 identity 的 spatial-frequency update，再将它作为真正独立的 Route C 送入三路融合；旧 v3/非三路配置仍调用兼容的 `forward(x)=x+forward_route(x)`，所以旧 checkpoint 行为和 state keys 不变。

内容感知融合。 对于第 r 条路的特征 $\mathbf{F}_r$ ，代码先做 LayerNorm2d 与 1×1 通道对齐，再同时根据全局通道描述子和局部空间特征预测 logits：

$$w_r(c, h, w) = \text{softmax}_r(g_r^{\text{channel}}(c) + g_r^{\text{spatial}}(h, w)), \quad \sum_r w_r = 1. \quad (6)$$

1, 1/2, 1/4 尺度融合 NAF 与 Heavy U；1/8 尺度融合 NAF、Heavy U 与 adaptive spatial-frequency。瓶颈后只使用两层原生 PyTorch scaled-dot-product attention 做 Restormer 式转置通道注意力，避免在 256^2 token 上构建昂贵的全局 Transformer。低分辨率 attention 是对全局交互的有界投资；ESC 对高效 attention/卷积替代的分析进一步支持“不在高分辨率机械堆标准 Transformer”的工程选择 [17]。

融合不使用严格零初始化，而是以 $\alpha = 0.05$ 小残差起步；频率内路尺度初值为 1，稳定性由外层 α 统一控制。这避免“内层尺度等于 0，外层尺度也等于 0”导致分支前几步完全收不到梯度。实际 BF16 forward/backward 检查已确认 Heavy U、adaptive frequency、fusion 和 context 都有非零梯度。

日志如何证明分支真的参与。 `RestorationOutput.fusion\_diagnostics` 每次前向返回四个尺度的 route gate 均值、归一化熵、Heavy/Main 与 Frequency/Main 范数比、独立 fre-

quency route/input RMS 比、融合尺度，以及 adaptive cutoff、transition 和 low/high modulation gate。Trainer 不会每 batch 调用 `item()` 强制 GPU 同步；它在设备上用 float32 累加，epoch 末统一写入 `log/<run>/epoch_metrics.jsonl` 和 `performance_summary.json`。这些值不是竞赛性能指标，而是诊断“门控是否塌缩到某一路”、“新分支是否一直近似零”的工程证据。

为什么不物理删除 Restormer、CAMP-VS v2 和 ensemble。这些文件在 v4 YAML 下不会被实例化，因此不占用 v4 的 GPU 显存或每轮计算时间；物理删除只会省下很小的源码空间，却会破坏旧 checkpoint 读取、配置回放、消融和回滚。ensemble 更不是单模型 forward 中的一个开关，而是至少两个独立 checkpoint 训练完成后的推理融合阶段。完整 Mamba 本轮也不进主线：它会引入 selective-scan CUDA 扩展、环境风险，而“理论 FLOPs 较低”不自动等于 4090 实际吞吐更高。

4.4 输入预处理与 Sobel 结构提示

当前配置启用 `use_sobel_input=true`。固定 Sobel 卷积核计算水平和垂直梯度，再取

$$S = \sqrt{G_x^2 + G_y^2} + \varepsilon, \quad (7)$$

得到边缘强度图。当前 Dataset 已把 RGB 存储文件转换为一个逻辑灰度通道，所以 Sobel 直接在这一通道上计算。原输入和 Sobel 图在通道维拼接成 2 通道，再通过保持 256×256 尺寸的 3×3 stem 卷积。Sobel 没有可训练参数，它只是明确告诉模型“哪里变化快”，类似在原图旁边附上一张铅笔勾边图。

4.5 NAF 局部编码器

NAF (Nonlinear Activation Free) 来自 NAFNet 图像恢复思想 [2]。名称的重点是：NAFBlock 主干不依赖 ReLU/GELU 这类常规逐点激活，而使用乘法门控建立非线性。它并不意味着整个模型完全没有激活函数；当前 task adapter 使用 GELU，prototype 分配使用 softmax，最终输出使用 sigmoid。

4.5.1 SimpleGate

若输入特征有 $2C$ 个通道，SimpleGate 沿通道一分为二：

$$\text{SimpleGate}([\mathbf{x}_1, \mathbf{x}_2]) = \mathbf{x}_1 \odot \mathbf{x}_2, \quad \mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^{C \times H \times W}. \quad (8)$$

$\odot$ 表示逐元素相乘。直观地说，一半特征像“内容”，另一半像“开关强度”，两者相乘后共同决定信息是否通过。它没有额外参数，但乘法本身产生了非线性关系。

4.5.2 SCA 简化通道注意力

SCA (Simplified Channel Attention) 先对每个通道做全局平均池化, 得到 $C \times 1 \times 1$ 的摘要; 再用 1×1 卷积生成通道缩放系数, 并乘回原特征:

$$\text{SCA}(\mathbf{x}) = \mathbf{x} \odot \text{Conv}_{1 \times 1}(\text{GAP}(\mathbf{x})). \quad (9)$$

它比包含多层全连接和 sigmoid 的传统 SE 模块更简单。这里的“注意力”可理解为模型在问: 当前图像中, 哪些特征通道更值得放大?

4.5.3 NAFBlock 的真实顺序

图 10 按 `naf_blocks.py` 展示两个残差子块。第一支负责空间混合与通道重标定, 第二支像门控前馈层。 β 与 γ 都从 0 初始化, 所以网络刚建立时每个 block 接近恒等映射, 有利于深层模型稳定开始训练。

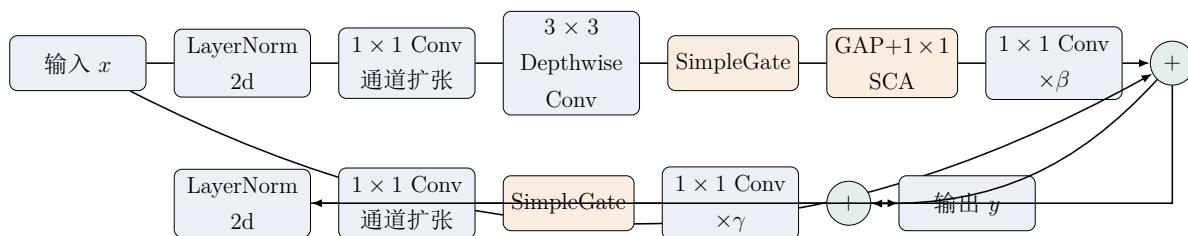


图 10: NAFBlock: 两次归一化、两条门控残差路径和零初始化残差缩放

4.5.4 四个编码尺度

`MultiMarkerRestorer._encode()` 的 stem 保持 256×256 , 每个 NAF stage 之后 (最后一级除外) 再用 2×2 、stride 2 卷积下采样。当前通道为 $[64, 128, 256, 512]$, depths 为 $[2, 2, 4, 6]$:

表 8: 局部编码器各尺度

相对尺度	空间尺寸	通道数	NAFBlock 数	主要作用
1	256×256	64	2	细边缘与局部纹理
1/2	128×128	128	2	中等范围结构
1/4	64×64	256	4	更大感受野与 skip
1/8	32×32	512	6	原型混合前的瓶颈表示

4.6 瓶颈处的共享/任务余弦原型混合

原型混合器本身没有被 Restormer 替换; 它仍调用 `MultiTaskPrototypeMixer.v4` 只在三路特征进入原型层之前增加两层低分辨率原生 PyTorch SDPA 精炼, 因此全局交互

和 shared/task prototype 是先后互补而非两套互斥主模型。随后把 $\mathbf{F} \in \mathbb{R}^{B \times 512 \times 32 \times 32}$ 展平后，每个空间位置成为一个 512 维 token，总计 1024 个 token。代码把 token、8 个 shared prototype 和 8 个 CD68 task prototype 都做 L_2 归一化，再计算余弦相似度和温度为 0.1 的 softmax:

$$A_{ik}^{(s)} = \text{softmax}_k \left(\frac{\bar{\mathbf{f}}_i^\top \bar{\mathbf{p}}_k^{(s)}}{0.1} \right), \quad A_{ik}^{(t)} = \text{softmax}_k \left(\frac{\bar{\mathbf{f}}_i^\top \bar{\mathbf{p}}_k^{(t)}}{0.1} \right). \quad (10)$$

shared 与 task attention 分别对原型加权求和；两个 context 拼接后经线性层投影回 512 维，再以可学习 fusion_scale 残差加回原特征。该尺度初始值为 0.1，所以原型分支从小幅修正开始，而不是一开始覆盖卷积特征。相似度、归一化和 softmax 在 float32 中计算，随后再转回输入 dtype。

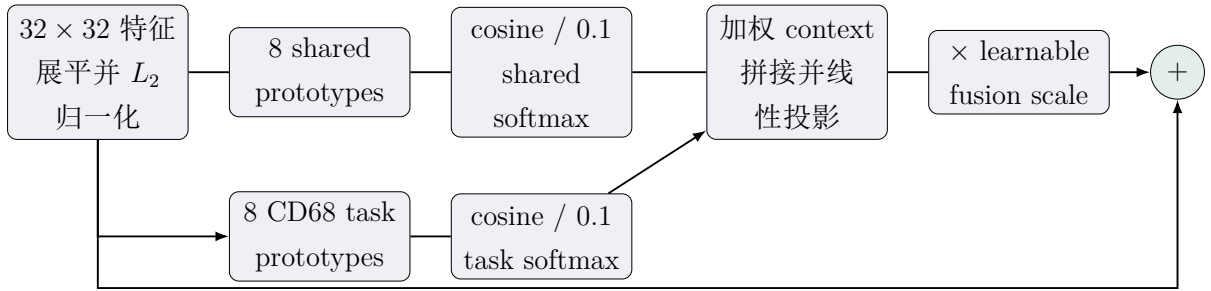


图 11: MultiTaskPrototypeMixer: 共享/任务原型注意力与残差融合

4.7 解码器与 skip connection

解码器有三个 DecoderStage。每一级先用双线性插值把低分辨率特征放大到 skip 的尺寸，再用 1×1 卷积调整通道，把放大特征和编码器 skip 在通道维拼接，经过另一个 1×1 卷积融合，最后用 NAFStage 精炼。skip connection 的作用像给“根据摘要重画原图”的人补充原始草稿：瓶颈保留大局，浅层 skip 带回精细边缘。

三个解码输出尺度分别是 64×64 、 128×128 和 256×256 ；每级 NAFStage 都有 2 个 NAFBlock。每一级随后可经 task adapter，并由独立 1×1 head 产生该尺度的 sigmoid 预测。

4.8 marker conditioning、FiLM 与 adapter

conditioning（条件控制）让共享模型知道当前要预测哪个 marker。当前主模型为目标建立 64 维 task embedding；它不调用 CAMP-VS v2 的 organ conditioner。**FiLM**（Feature-wise Linear Modulation）根据 embedding 生成逐通道缩放和偏置：

$$\text{FiLM}(\mathbf{F}) = (1 + \gamma) \odot \mathbf{F} + \beta. \quad (11)$$

解码器每级的 TaskAdapter 先由 embedding 生成 scale/shift，线性层从零初始化；再经过 LayerNorm2d、depthwise 3×3 、GELU 和 1×1 pointwise 路径，残差尺度也从 0 开

始。当前只有 CD68 一个任务，embedding 不能在多个 marker 之间做选择，但接口保留了以后多任务扩展能力。

4.9 direct output 与零初始化 base/detail residual

v3/v4 都保留每个解码尺度的 direct 1×1 output head；当前 v4 在最终 256×256 尺度开启 LaplacianBaseDetailHead 的 residual-to-reference 模式。direct 路径仍为

$$\hat{\mathbf{y}}_s = \sigma(\text{Conv}_{1 \times 1}(\mathbf{F}_s)), \quad s \in \{64, 128, 256\}. \quad (12)$$

最终尺度再按上一节公式加入零初始化 coarse/detail residual。全局强度标定器 GlobalIntensityCalibrator 仍未启用，因为本轮没有验证证据表明它能稳定提高 JPG 指标。

4.10 Deep Supervision

深监督（Deep Supervision）是在中间解码层也产生预测，让浅层和深层都直接收到训练信号。当前只有三个输出尺度：256、128、64。目标图用 area 插值缩放到对应大小。损失权重在训练中从 [1.0, 0.5, 0.25] 平滑退火到 [1.0, 0.1, 0.05]：前期让中间层容易获得梯度，后期把容量集中到真正用于提交的全分辨率输出。

4.11 当前实际模型规模

结构统计区分稳定 v2 对照、已停止的 v3 和当前 v4 候选。v2 为 16,232,644 参数、约 39.160 GMAC；v3 为 18,074,199 参数、约 44.11 GMAC；v4 的真实构造结果为 29,130,894 参数、约 75.997 GMAC/单张 256×256 forward。MAC 只统计可计数卷积/线性运算，不完整包含 FFT、SDPA attention matmul、SSIM、JPEG 解码、15% batch 的第二次 D4 forward/backward 或九张邻域图像的 I/O，因此不能直接换算成小时数。

表 9: 稳定对照、v3 与 v4 候选的结构规模

配置	参数量	约 GMAC/图	状态
retrain-v2 稳定对照	16,232,644	39.160	已完成训练/验证
max-v3 已停止候选	18,074,199	44.11	用于结构对照
其中轻量 Detail U	72,016	0.589	v3 已启用
max-v4 当前候选	29,130,894	75.997	工程 smoke 通过，待 4090 训练
其中 Heavy Omni U	4,509,672	包含于 v4	v4 已启用
多尺度 + 三路融合	6,453,341	包含于 v4	v4 已启用

v4 当前启用 Sobel、NAF、Heavy Omni U、adaptive spatial/frequency、四尺度 gate、1/8 tri-path SDPA、verified 3×3 ROI context、9-token cross-attention、shared/task prototypes、CD68 adapters 和 residual base/detail。故意关闭的模块包括与单目标不相容的 FAMO/correlation、与零配准偏移冲突的 shift-tolerant loss、没有正收益证据的 foreground loss，以及会显著扩大时间/依赖风险的完整 Mamba、扩散和 GAN。这里的“全开”指所有与单目标、已对齐、9 小时预算相容的候选模块，而不是机械打开互相冲突的开关。

5 数据流向详解

5.1 用一个真实 patch 对照源码路径

图 12 继续使用 ROI007_09_11。上方蓝色箭头表示 DAPI 真正进入模型的路径；下方红色分支表示训练时才会读取的 CD68 真值。图中的 CD68 小图是训练标签，不是模型预测。当前报告构建目录没有完成正式训练的 competition checkpoint，因此本文不会拿真值冒充“生成结果”。真正的预测外观必须由指定 checkpoint 在 test DAPI 上运行后产生。

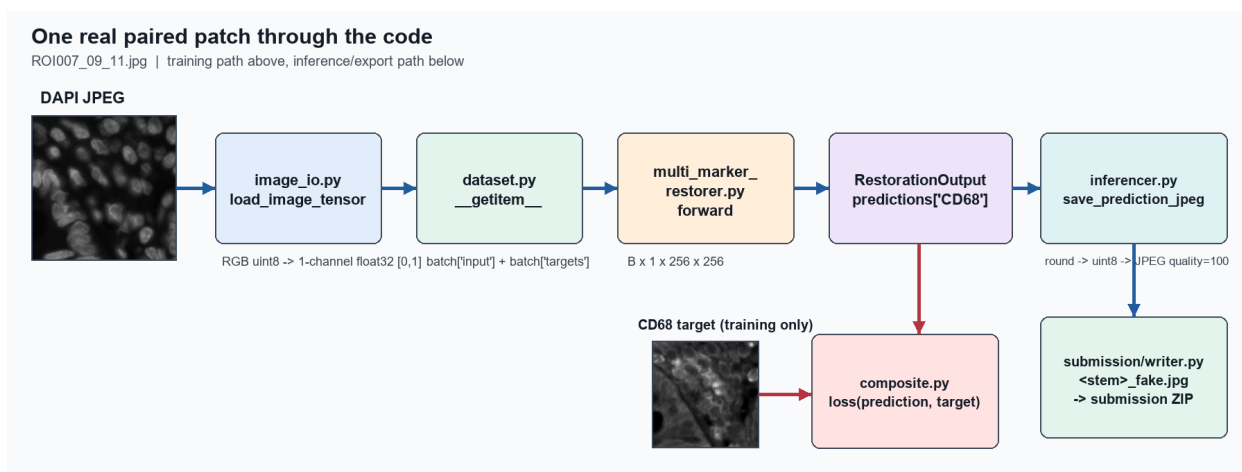


图 12: 一个真实配对 patch 从 JPEG、Dataset、MultiMarkerRestorer 到损失与提交文件的代码路径。

表 10 把图中的每个方框与当前源码对应起来。需要特别注意：四个 target 文件夹不会同时“喂给”单目标 CD68 模型。配置中的 `data.targets` 决定 Dataset 读取哪些真值，`model.name` 决定 CLI 构造哪一个网络；当前初赛配置只读取 CD68 作为监督目标，并构造一个 MultiMarkerRestorer 实例。

表 10: 真实 patch 在代码中的表示变化

阶段	输入/数据键	对应源码	产生的结果
图像解码	DAPI/CD68 JPG	utils/image_io.py 的 load_image_tensor()	RGB uint8 HWC 的三个相同通道先取均值并四舍五入, 变为 $[0, 1]$ 的单通道 float32 CHW。
样本组装	manifest 一行	data/dataset.py 的 VirtualStainingDataset. __getitem__()	返回 batch["input"]、batch["targets"]["CD68"] 和 ROI/stem 元数据。
选择模型	合并后的 YAML	cli.py 的 _build_model()	当 model.name=multi_marker_restorer 时实例化当前 v4 MultiMarkerRestorer; 不会另外实例化 CAMP-VS v2 或独立 Residual U-Net。
前向传播	$B \times 1 \times 256 \times 256$ DAPI 与 9 个 context tiles	models/multi_marker_restorer.py 的 MultiMarkerRestorer. forward()	NAF、Heavy U、自适应 FFT 分别提取特征, 经四尺度门控、低分辨率 SDPA、context、原型与 task-adapted 解码后返回 RestorationOutput.predictions["CD68"]。
训练比较	预测图与 CD68 真值	losses/scheduled_composite.py、losses/competition_proxy.py 和 engine/trainer.py	两阶段重建/结构损失、深监督退火、后段 float metric proxy 与可选 D4 等变约束; CD68 真值只走监督分支, 不作为模型输入。
推理保存	test DAPI 与 checkpoint	engine/inferencer.py、 submission/writer.py	预测先保存为 <stem>.jpg, 提交构建时命名为 <stem>_fake.jpg 并写入 ZIP。

从神经网络内部再向下看, MultiMarkerRestorer.forward() 不是直接对每个像素套一个公式。它先调用 _encode() 得到四尺度 features, 在瓶颈执行 shared/task prototype mixing, 再按任务调用 _decode_task()。CD68 分支产生主预测、三尺度深监督和 prototype attention 诊断张量, 最后装入 RestorationOutput。训练器取预测与真值算 loss; 推理器只取预测, 执行可选 D4 平均、截断到 $[0, 1]$ 、四舍五入到 uint8, 并以 quality 100、subsampling 0 编码一次 JPEG。提交 writer 随后直接复制已有 JPEG 并改成比赛文件名, 避免第二次有损编码。

5.2 训练数据流

图 13 把训练过程从硬盘一直画到参数更新。张量形状按当前单逻辑通道配置表示。

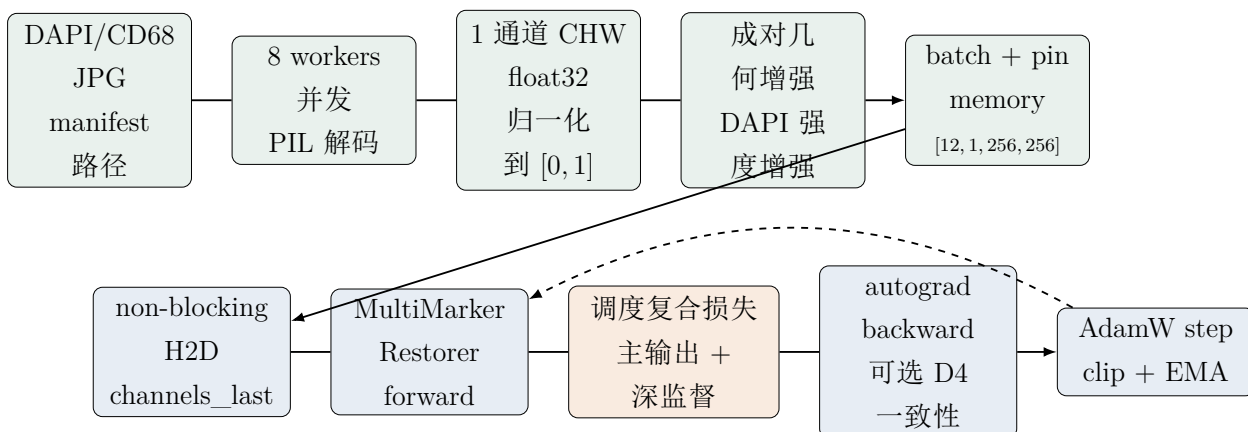


图 13: 训练时从 JPG 到模型参数更新的数据流

进一步拆解如下：

1. AutoDL v4 默认由 8 个 persistent workers 按 manifest 预取配对 JPG，PIL 解码为 RGB uint8；Windows 可显式覆盖为 0。
2. `load_image_tensor()` 对三个相同 RGB 通道取均值并四舍五入为一个逻辑通道，再转为 CHW float32 并除以 255。
3. PairedTransform 同步改变 DAPI/CD68 的几何，仅扰动 DAPI 强度。
4. DataLoader 把 6 个样本堆成物理 batch；每累积 2 个 batch 才更新一次参数，所以 effective batch 仍为 12。页锁定内存为 CPU 到 GPU 传输做准备。
5. `move_to_device()` 对张量使用 non-blocking CUDA copy；训练器在启用时把 4D 浮点输入转为 channels-last 物理布局。
6. autocast 选择 BF16 或 FP16 执行适合低精度的算子；模型返回结构化预测结果。
7. scheduled composite loss 将预测与 CD68 真值比较，连续调度均方误差、稳健像素误差和 SSIM 等权重，并把 256/128/64 三个分辨率的退火深监督合并；后段再加入小权重 float metric proxy。
8. 正常每个 DataLoader batch 执行一次反向传播、梯度累积、优化器更新和 EMA 更新；v4 的 15% batch 另计算一次随机 D4 等变一致性。CUDA OOM 时只在 batch 内切 chunk，不改变有效 batch。

5.3 验证数据流

验证不更新模型。它要回答的问题是：“用当前权重预测从未用于梯度更新的 ROI，文件质量怎么样？”

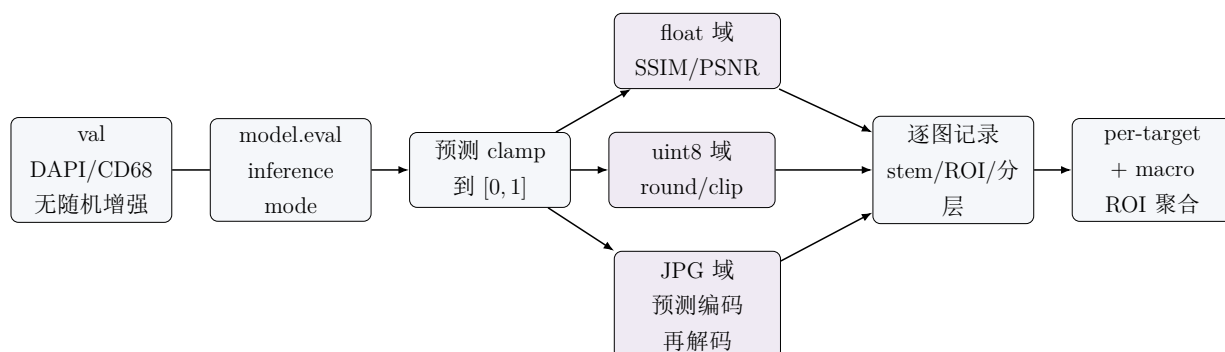


图 14: 验证时的三域指标与 ROI 聚合数据流

验证 loader 不创建 PairedTransform；模型使用 `eval()` 和 `torch.inference_mode()`，所以 dropout 等模块进入推理行为，也不保存梯度图。每张预测先截断到 $[0, 1]$ ，再进入三个指标域。结束后恢复模型原来的 train/eval 状态，避免验证永久改变训练模式。

5.4 测试与推理数据流

测试 manifest 只有 DAPI。Inferencer 保证 loader 不能随机打乱，然后按稳定顺序预测。当前正式提交配置为 `tta=d4`、`tta\_policy=configured`，因此配置中的 D4 是权威选择，不会被一次不完整的自动晋级审计静默关闭。同一 batch 会经历恒等、 90° 、 180° 、 270° 、水平翻转、垂直翻转、转置和反转置八种变换。每个预测经过精确逆变换后在 float32 中求平均：

$$\hat{\mathbf{y}}_{D4} = \frac{1}{8} \sum_{k=1}^8 T_k^{-1} (f_{\theta}(T_k(\mathbf{x}))). \quad (13)$$

平均结果 round 到 uint8，并按 checkpoint ImageSpec 指定的 RGB 模式，以 quality 100、subsampling 0、optimize false 编码一次 JPEG，先保存为 `predictions/CD68/<stem>.jpg`。提交构建器再把它直接复制成 `results/test/CD68/<stem>_fake.jpg`，不会把 JPEG 再解码重编码。

推理也有最多三次定向 CUDA OOM 降级：发生明确 CUDA out-of-memory 时把 batch 切成更小 chunk；其他异常不会被误当 OOM 吞掉。

6 训练详解

6.1 epoch 与 batch: 把“看很多遍数据”说清楚

epoch (轮次) 表示把当前训练集完整遍历一次。当前有 5004 个训练 patch, v4 物理 batch size 为 6, DataLoader 不丢弃最后不足 6 张的 batch, 因此每个 epoch 大约有

$$\left\lceil \frac{5004}{6} \right\rceil = 834 \quad (14)$$

个 batch。当前配置最多训练 100 epoch, 不代表“只看 100 张图”, 而是反复遍历数据 100 次, 并且每次都可能得到新的随机增强。

`Trainer.fit()` 是外层 epoch 循环, `train_epoch()` 是内层 batch 循环。每个 epoch 开始时模型进入 train 模式、清空梯度、重置显存峰值统计; 结束时记录真实耗时、吞吐量、峰值显存、RSS 内存、学习率、OOM 次数和损失分量。

6.2 一次 batch 内发生什么

对第 t 个 batch, 训练器完成:

1. `unpack_batch()` 得到输入、目标字典和元数据;
2. 张量移动到 GPU, 并按配置转 channels-last;
3. 在 autocast 中执行模型 forward 与 loss;
4. 检查总 loss 必须是一个有限标量, 防止 NaN/Inf 污染参数;
5. 将 loss 除以梯度累积因子并 backward;
6. 到达累积边界后进行梯度裁剪、AdamW step、EMA 更新, 并以 set-to-none 方式清空梯度。

6.3 梯度累积与 effective batch

当前 `batch_size=6`、`gradient_accumulation=2`, 所以正常情况下

$$B_{\text{effective}} = 6 \times 2 = 12. \quad (15)$$

每个 epoch 正常约有 834 个 DataLoader batch、417 次 optimizer step, EMA 也大约更新 417 次。若 24 GiB 4090 发生明确 CUDA OOM, 训练器会把同一个 DataLoader batch 内部分成 3+3 等 chunk; 各 chunk 的梯度按真实样本数占原 batch 的比例缩放后再做一次 optimizer step, 所以有效 batch 仍是 12, 不会因为降级偷偷变成 24。

6.4 AMP 混合精度

AMP (Automatic Mixed Precision, 自动混合精度) 让卷积和矩阵乘等适合的算子使用 BF16/FP16, 对需要稳定范围的算子保留 float32。这样通常可以减少显存并利用 Tensor Core 加速 [7]。

当前 v4 显式设置 `amp=true`、`amp_dtype=bfloat16`, 面向 RTX 4090/5090 使用 BF16 Tensor Core。GradScaler 只在 CUDA+FP16 时启用; BF16 的指数范围较大, 代码不启用 scaler。SSIM、FFT、Sobel gradient 和 prototype cosine 等敏感计算会显式升到 float32, 这说明“开 AMP”不是把每个数都粗暴降精度。

6.5 AdamW、学习率、warmup 与梯度裁剪

当前优化器是 AdamW, 初始学习率 1.8×10^{-4} , weight decay 10^{-4} 。AdamW 将 weight decay 与自适应梯度更新解耦 [5, 6]。前 4 个 epoch 是 warmup, 学习率从较小倍数逐步升到基础值; 之后使用余弦函数下降。v4 还把 bias、normalization、NAF beta/gamma 和 residual scale 放入 no-decay 参数组; CUDA 支持时使用 fused AdamW, 不支持则自动回退普通 AdamW:

$$\eta(p) = \eta_0 \frac{1 + \cos(\pi p)}{2}, \quad p \in [0, 1]. \quad (16)$$

前 4 个 epoch 的 warmup 像汽车起步时缓慢踩油门, 避免随机初始化阶段更新过猛。每次 optimizer step 前, 所有优化参数的梯度总范数被裁剪到 1.0, 防止个别 batch 产生异常大的更新。

6.6 EMA: 不是另一个独立模型

EMA (Exponential Moving Average, 指数移动平均) 保存一份参数影子:

$$\bar{\theta}_t = 0.999\bar{\theta}_{t-1} + 0.001\theta_t. \quad (17)$$

它不是再训练一遍模型, 而是在每个 optimizer step 后平滑跟踪当前参数。验证 EMA 时, 代码暂时备份 raw 权重、把 EMA shadow 复制进模型、完成验证后恢复 raw。短训练时 EMA 可能严重滞后, 所以当前训练配置明确在每个验证 epoch 同时评价 raw 和 ema, 按 JPG SSIM、再按 JPG PSNR 选择更好的来源, 而不是先验认定 EMA 一定更优。选中来源会写入 checkpoint 的 extra 元数据, 提交时 `weight_source=best_jpg` 会读取它。

6.7 稳定 v2 对照的固定权重复合损失

只用像素误差可能得到模糊结果, 只用结构指标又可能忽略强度与数值误差。稳定 v2 对照的 `loss.schedule=constant`, 所以 `_build_loss()` 构造 `CompositeRestorationLoss`, 不会调用 two-phase 的 `ScheduledCompositeLoss`。每个输出尺度组合五项重建损失:

$$\mathcal{L}_{\text{Charb}} = \frac{1}{N} \sum_i w_i \sqrt{(\hat{y}_i - y_i)^2 + \varepsilon^2}, \quad (18)$$

$$\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(\hat{\mathbf{y}}, \mathbf{y}), \quad (19)$$

$$\mathcal{L}_{\text{freq}} = \|\log(1 + |\mathcal{F}(\hat{\mathbf{y}})|) - \log(1 + |\mathcal{F}(\mathbf{y})|)\|_1. \quad (20)$$

其中 w_i 是由目标局部均值差异构造的 structure weight；另外还有 MS-SSIM 和 Sobel gradient。Charbonnier 是平滑的 L1 近似，gradient 强调边缘，frequency 比较 FFT 的 log amplitude 而不直接约束容易不稳定的相位。

表 11: 稳定 v2 对照的固定损失重建项权重

分量	权重	直观作用
Charbonnier	0.40	带 structure weight 的稳健像素重建
SSIM	0.35	局部结构、亮度与对比度
MS-SSIM	0.10	多尺度结构
Gradient	0.10	Sobel 边缘一致
Frequency	0.05	频谱幅度和纹理能量一致

三个 deep-supervision 尺度各自计算上述五项，再按 $[1.0, 0.5, 0.25]$ 相加。MSE、pyramid 与 statistics 权重在有效配置中均为 0。跨 marker correlation 的配置权重虽为 0.02，但当前只有 CD68 一个 prediction，`task_correlation_loss()` 按定义返回图连接的 0，不会贡献梯度。

prototype activation 与 diversity 两项则**真实启用**，权重各为 0.001。前者让采样到的瓶颈 token 靠近至少一个 shared/CD68 prototype，后者惩罚同一 bank 内原型间的平方余弦相似度，降低所有原型坍缩到同一方向的风险。因此总损失可以概括为

$$\mathcal{L} = \sum_{s \in \{256, 128, 64\}} \lambda_s \mathcal{L}_{\text{recon}, s} + 0.001 \mathcal{L}_{\text{proto-act}} + 0.001 \mathcal{L}_{\text{proto-div}}. \quad (21)$$

6.8 v3/v4 两阶段损失、指标对齐与 D4 等变

v3 引入、v4 继承 `ScheduledCompositeLoss`。训练进度 $p \in [0, 1]$ 从 phase A 平滑过渡到 phase B：前者以 Charbonnier 和多尺度结构为主，后者提高 MSE/SSIM 对最终像素指标的约束，并以小权重加入 Laplacian pyramid、gradient、statistics 与 log-frequency。连续 cosine 插值避免在阶段边界突然改变梯度方向。三个深监督权重也按同一进度从 $[1, 0.5, 0.25]$ 退火到 $[1, 0.1, 0.05]$ 。

从训练后 55% 开始，最终 256×256 输出还加入权重上限 0.06 的 float-domain metric proxy。它用与 scikit-image float SSIM 一致的 uniform 7×7 、sample covariance 和 valid crop，并按每图 MSE 计算 capped PSNR loss。它**不模拟** uint8 round 或 JPEG 编解码；真

正 checkpoint 选择仍以非可微的 JPG round-trip 验证为准，不能把这项 loss 称为“可微 JPEG”。

训练进度超过 10% 后，D4 consistency 权重平滑升至 0.015；v4 在 15% 的 batch 随机抽取一个非恒等 D4 变换，同步旋转/翻转中心图、九张 context tiles、valid mask 和二维 offsets，再最小化

$$\mathcal{L}_{eq} = \text{SmoothL1} \left(T^{-1} f(Tx), \text{stopgrad}(f(x)) \right). \quad (22)$$

这会平均增加约 15% batch 的额外 forward/backward，但不增加推理参数。末段 hard-mix sampler 只在最后 22% epoch 把 30% 槽位温和分给最高 activity 桶；总采样数不变，test 永远不参与训练。

6.9 channels-last、cuDNN benchmark 与 float32 matmul precision

当前配置为 CUDA 训练启用 channels-last。逻辑张量形状仍写成 NCHW，但物理内存更接近 NHWC 排列，卷积密集模型可能更容易走 Tensor Core 友好的内核。训练模型和输入都会转换；Validator 也转换输入。当前 Inferencer 没有对应的 channels-last 输入转换路径，因此不能把训练/验证优化自动外推到最终推理。

deterministic=false、cudnn_benchmark=true 允许 cuDNN 针对固定 256×256 尺寸测试并选择较快卷积算法，代价是严格逐位复现能力下降。float32_matmul_precision=high 在支持的 NVIDIA GPU 上允许 float32 矩阵乘采用更快的内部精度路径（通常包括 TF32）。这些是性能设置，不是保证每台 GPU 都获得相同比例加速的魔法开关。

6.10 进度条、异步指标、profiler 与 torch.compile

- `progress_bar=auto`: 仅在交互式终端的 stderr 是 TTY 时显示 tqdm batch 进度，内容包括 loss、图/秒、学习率和峰值显存；nohup 重定向时默认关闭，避免日志被刷新字符污染。
- `async_metric_logging=true`: loss 分量留在 GPU 累积，epoch 末统一同步，避免每个 batch 因转 Python float 让 GPU 等待 CPU。
- 验证按 pass 输出静态进度：训练期 raw/EMA 分别标为 `pass=1/2`、`pass=2/2`；训练后的普通与 D4 验证也会打印开始、完成、样本数和耗时，不会在长时间 JPG round-trip 中完全沉默。
- `gpu_monitor.enabled=true`: 后台每 2 秒调用一次 `nvidia-smi`，把 GPU utilization、显存、功耗和温度的 mean/p95/max 汇总进 epoch 日志；监控失败只丢诊断，不中断训练。
- `profiler.enabled=false`: 可选 profiler 能跟踪最初若干 batch 并导出 Chrome trace，但当前生产训练不承担这项开销。

- `compile.enabled=false`: 可选 `torch.compile` 失败会回退 `eager`; 当前配置没有启用, 也不存在“已经通过 `compile` 加速”的结果。

6.11 OOM 恢复

只有错误明确属于 CUDA out-of-memory 时才进入恢复: 清梯度、清缓存, 把同一个 DataLoader batch 在内部切成更小 chunk。配置的 **gradient accumulation** 不再倍增; 若 OOM 清除了一个尚未完成的累积窗口, 窗口计数会从重试成功的 batch 重新开始。最多三次; chunk 已经为 1 仍 OOM 时抛出错误。这样能保持原有效 batch, 又不会把文件损坏、shape 错误等普通 bug 伪装成显存问题。虽然 YAML 有 `max_oom_retries=3`, 当前 Trainer 实现实际把“三次”直接写在判断逻辑中。

6.12 验证频率、checkpoint 与早停

v4 有效 `validate_every_n_epochs=5`, 所以每 5 个 epoch 验证一次; 最后一个 epoch 仍会强制验证; 跳过验证的 epoch 只更新 `last.ckpt`, 不会冒充 best 或 top-k。

每个 epoch 原子写入 `last.ckpt`。验证指标创新高时分别写 `best_ssim.ckpt` 与 `best_proxy.ckpt`。当前 `save_top_k=0`, 不额外保留重复 top-k 大文件。checkpoint 版本 2 包含模型、EMA、优化器、scheduler、scaler、loss 状态、epoch、global step、完整配置、manifest hash、ImageSpec、目标列表、指标历史、随机数状态和 DataLoader generator 状态, 因此 resume 不是“只加载一份权重重新开始”。

early-stopping patience 为 5, 计数单位是**实际验证事件**; 即连续 5 次三域验证 (最多覆盖 25 个 epoch) 没有改进时可以提前停止。停止时日志、history 与摘要都会写入 `stop_reason=early_stopping`、当前/最佳指标和未提升次数, 不再让用户猜测为何没有跑到 100。此外, `max_wall_time_hours=8.5` 使用最近 epoch 耗时预测到下一个安全验证点, 只在已验证且 best/last checkpoint 完整写入后停止。它是训练阶段的预测保护, 不是操作系统硬截止; 数据审计与最终 D4 仍要另留时间, 所以报告只给出 4090 整条命令目标约 9 小时的预算估计, 不冒充实测。

7 验证详解

7.1 训练和验证的根本差异

训练需要随机增强、梯度和 optimizer; 验证不增强、不建梯度、不开 optimizer。验证数据按 ROI 与训练集隔离, 因此分数衡量的是模型对未见 ROI 的泛化, 而不是对训练样本的记忆。

稳定 retrain-v2 的 AutoDL 日志已经提供完整 1292 张 val 证据: EMA 无 TTA 的 JPG SSIM/PSNR 为 0.804480/25.97599, EMA+D4 为 0.809280/26.19677; 整条命令耗时 3 小时 25 分, 训练部分 2 小时 45 分, 峰值显存 11.63 GiB。v4 尚未在 4090/5090 上完成正式

训练，因此这些数字只能作为固定对照，不能写成 v4 已涨分，更不能伪造平台 test 指标。v4 的实际数字必须由附录中的命令真实生成。

7.2 SSIM 与 PSNR

代码用 scikit-image 对每张图单独计算。SSIM 的经典形式 [4] 为

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}. \quad (23)$$

μ 比较平均亮度， σ 比较对比度， σ_{xy} 比较结构共同变化。项目数据范围为 1。PSNR 从均方误差得到：

$$\text{MSE} = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2, \quad \text{PSNR} = 10 \log_{10} \frac{1^2}{\text{MSE}}. \quad (24)$$

若预测与目标完全相同，MSE 为 0，PSNR 合法地为正无穷；聚合代码会记录无穷值数量。

7.3 float、uint8 与 JPG 三域

float 域 直接比较 $[0, 1]$ 浮点预测与解码后的浮点目标。它反映模型张量本身的理想质量。

uint8 域 预测和目标都按与保存函数相同的 round+clip 规则量化为 $0 \sim 255$ ，再除以 255 评分。它衡量 8 位量化损失。

JPG 域 只把预测按 ImageSpec 的 RGB 存储模式、quality 100、subsampling 0、optimize false 编码到内存，再解码；目标已经来自官方 JPEG，只做 uint8 量化，不再次压缩。它衡量预测最终落盘可能引入的额外误差。

初赛配置设置 `primary_domain=jpg`。这是很合理的本地模型选择代理，因为提交文件就是 JPEG；但在没有赛事平台评测代码的情况下，不能写成“官方评测器一定逐字逐步完全相同”。正确说法是：JPG 域更接近最终提交文件形态，官方隐藏 test 分数仍只能由上传 ZIP 后的平台给出。

7.4 逐图、ROI、最差样本与分层聚合

每条验证记录包含 target、stem、ROI、organ、坐标、context 可用度、目标图平均值/标准差、activity proxy 和三域 SSIM/PSNR。聚合层输出：

- 图像级 mean/median SSIM、PSNR；
- 每个 ROI 内先平均、再在 ROI 间宏平均的 `roi_ssim/roi_psnr`；
- 最差 10% 图像的均值，用于发现模型是否只照顾容易样本；

- 按 target 的 per-target 指标与跨 target macro;
- 按 organ、activity、context availability、图像平均亮度、border/interior 的分层结果;
- 本地 proxy: $0.7 \times \text{SSIM} + 0.3 \times \text{截断归一化 PSNR}$ 。

当前 manifest 的 organ 是 unknown, 因为目录是 train/marker 而不是带器官层级的目录; 所以不要把本地结果描述为 colon/liver/stomach 加权评测。坐标解析成功时, Validator 用每个 ROI 实际出现的最小/最大行列标记 border, 其余为 interior。

7.5 bootstrap 的真实状态

bootstrap (自助重采样) 通常指反复有放回抽样, 以估计指标差的置信区间。项目的 roi_bootstrap_difference() 会先在每个 ROI 内平均, 再对 ROI 做成对重采样, 默认 1000 次, 输出候选减基线的均值、95% 区间和 win/tie/loss。

但是当前 Validator.evaluate() 虽然读取了 YAML 中 bootstrap_by_roi=true 的意图, 却不会在单次普通验证报告中直接生成 bootstrap 区间。bootstrap 真正用于 TTA/promotion 比较路径, 例如 command_validate() 的 D4 promotion 和性能 v2 消融门。因而“普通 metrics.json 已经包含 1000 次 bootstrap CI”是不准确的。普通验证包含 ROI 聚合; 候选与基线的置信区间需要走 promotion 比较流程。

7.6 raw、EMA 与 D4 的选择

训练期验证对 raw、EMA 分别完整评分, 然后按 JPG macro SSIM/PSNR 选更优来源。独立 command_validate() 加载 checkpoint 时, best_jpg 会读取这个来源; 它不是再次自动跑 raw+EMA 两套。若 inference.tta=d4, 命令还会运行无 TTA 与 D4, 并利用同一 ROI 清单做 promotion 判断, 把每个 target 的决策写入 validation/tta_decisions.json。预测时仅在 inference.tta_policy=validation_decision 时按 target 应用该文件; 正式 CD68 配置采用 tta_policy=configured, 因此配置中的 D4 为权威设置, 不会被不完整的 promotion 审计覆盖。本次 seed-2026 最佳 checkpoint 的 JPG 指标由无 TTA 的 0.804480/25.97599 提升至 D4 的 0.809280/26.19677, 因而正式提交默认执行 D4。

8 测试与提交详解

8.1 为什么 test 不能显示本地分数

test 只有 DAPI, 没有 CD68 真值。SSIM/PSNR 都属于 full-reference metric, 必须同时有预测和真值。因此运行 autodl-submit 能得到 1346 张预测和合法 ZIP, 却不能得到 test 分数。平台持有隐藏 CD68 标签, 上传后才返回 official score。把 val 分数叫“test 分数”或用 val 标签冒充 test 都会误导实验判断。

8.2 D4、权重来源与 JPEG 保存

当前推理使用以下四项设置：`use_ema=false`、`weight_source=best_jpg`、`tta=d4`，以及 `tta_policy=configured`。其中 `best_jpg` 优先使用 checkpoint 记录的训练期 raw/EMA 胜者；如果旧 checkpoint 没有记录，才根据 `use_ema=false` 回退到 raw，且 checkpoint 没有 EMA 时也会安全使用 raw。本次 seed-2026 checkpoint 的训练期胜者是 EMA。完整 1292 张验证集上，D4 把 JPG SSIM 从 0.804480 提高到 0.809280，PSNR 从 25.97599 dB 提高到 26.19677 dB，且 5 个 ROI 的聚合变化均为正，因此正式提交默认执行八次 forward。这里的“提交 D4”与训练期的“D4 等变一致性”不是同一件事：前者对每张 test 图执行八次前向并平均，后者在 v4 的 15% 训练 batch 上追加一次随机变换的一致性约束。提交 D4 不会改变 checkpoint 参数，但训练期一致性会参与梯度更新。

预测 tensor 先 clamp、乘 255、四舍五入为 uint8，按目标 ImageSpec 保存。当前目标审计为 RGB，所以最终模式应为 RGB，而不是 L。

8.3 提交目录与命名

构建后的真实结构是：

```
outputs/initial_round_max_v4/<run-id>/submission/
|-- results/
|   |-- test/
|       |-- CD68/
|           |-- ROI025_00_00_fake.jpg
|           |-- ...
|           |-- <1346 predictions>
|-- submission_CD68.zip
```

提交命名函数会先剥掉 stem 末尾已有的 `_fake`，防止产生 `_fake_fake.jpg`。ZIP 中每个成员必须从顶层 `results/` 开始，不能多包一层任意文件夹。

8.4 validate-submission 检查什么

validator 从 test manifest 推导 1346 个期望文件名，与实际目录做集合比较，并逐文件检查：

- 缺失、额外和重复输出；
- 扩展名和 `_fake` 后缀；
- 尺寸是否与 manifest 的 256×256 一致；
- JPEG 格式、uint8 dtype、RGB/L mode 和通道数；
- 像素范围、NaN/Inf、全黑和全白；近乎常量图给 warning；

- ZIP 成员是否都在 `results/` 下，是否缺/多文件，CRC 是否通过。

校验成功只代表“提交格式正确”，不代表“模型分数高”。格式校验像考试前检查答题卡是否填全；真正得分还要由平台拿隐藏答案批改。

9 配置系统：怎样读懂并安全修改 YAML

9.1 YAML 是什么

YAML 是一种供人阅读的层级配置格式。它把“经常调整的参数”从 Python 源码中拿出来，避免每次实验都改程序。例如：

```
train:
  epochs: 100
  batch_size: 6
  gradient_accumulation: 2
model:
  context:
    enabled: true
```

缩进表示所属关系，不能随意混用 Tab。这里 `train.epochs` 表示最大训练轮数，`model.context.enabled=true` 表示请求启用邻域上下文；v4 还要求 `require_verified_grid=true`，审计失败时不会静默猜邻居。布尔量应写成 `true/false`，列表可以写成 `[64, 128, 256, 512]`。

9.2 配置合并优先级

`load_config()` 不是只读一个文件，而是按下列顺序合并；越靠后的值优先级越高：

1. `configs/default.yaml`：所有实验共享的默认值；
2. 用户指定的 YAML，例如 `configs/initial_round_cd68_max_v4.yaml`；
3. `configs/resolved_local.yaml`：当前文件存在，保存本地审计推导出的数据根、逻辑通道和 worker 设置；
4. CLI 的 `--set key=value`：只覆盖这一次命令，优先级最高。

每个 run 都会把最后生效的配置写入输出目录。复现实验时应保存这个 effective config，而不应只凭记忆记录“我大概用了哪个 YAML”。例如临时跑 3 epoch 可以写：

```
python -m virtual_staining.cli \
  --log-root log \
  autodl-run \
  --config configs/initial_round_cd68_max_v4.yaml \
```

```
--run-id cd68_max_v4_quickcheck \
--max-epochs 3
```

--max-epochs 3 是快速检查，不是正式训练。必须换一个新的 run ID，避免把快速实验和 100 epoch 的正式实验混在一起。

9.3 当前初赛配置逐项说明

表 12 汇总当前真正影响初赛 CD68 训练的主要设置。

表 12: initial_round_cd68_max_v4.yaml 的关键生效配置

配置项	当前值	含义与影响
data.root	AUTO → dataset/official	自动发现官方 train/test; resolved-local 仍可覆盖。
data.targets	CD68	当前训练只学习 DAPI 到 CD68。
data.num_workers	8	AutoDL 使用 8 个 persistent workers、prefetch 4; Windows 本地可覆盖为 0。
logical channels	1 入 / 1 出	RGB 文件内容按审计结论压成一个逻辑灰度通道，提交时再恢复 RGB storage。
model.name	multi_marker_restorer	使用四尺度 NAF 主干及 v4 重型三路融合。
channels/depths	[64,128,256,512] / [2,2,4,6]	控制四级 NAF encoder 容量; stem 保持全分辨率。
heavy U	[56,88,128,176]	四尺度完整编解码，局部/轴向大核/空洞卷积，编码深度 [3,4,6,6]。
spatial/frequency	自适应开启	1/8 根据 patch 学习 FFT low/high 分界与双向调制。
multi-route fusion	四尺度开启	高分辨率 NAF/Heavy 门控; 1/8 NAF/Heavy/frequency softmax + 2 SDPA。
context	verified 3 × 3	tiny shared encoder; 1/4, 1/8 FiLM 与 9-token cross-attention。
prototypes	开启, 8 shared + 8 task	在 32 × 32 瓶颈执行余弦注意力与残差融合。
task adapters	开启	每个解码尺度使用 CD68 embedding 与零初始化 FiLM/residual adapter。

配置项	当前值	含义与影响
base/detail	direct + zero residual	coarse/detail 零初始化叠加旧 direct logits; calibrator 关闭。
loss schedule	two-phase	MSE/Charbonnier/SSIM 等连续调度, 深监督退火, 末段 float metric proxy。
train.epochs	100	最大轮数; 另有 8.5 小时安全停点预测保护。
batch / accumulation	6 / 2	有效 batch 仍为 12; 给重型激活留出 24 GiB 显存余量。
optimizer	grouped fused AdamW	学习率 1.8×10^{-4} 、weight decay 10^{-4} , bias/norm/residual scale no-decay。
scheduler	warmup+cosine	前 4 epoch 预热, 随后余弦衰减。
AMP	BF16	RTX 4090 Tensor Core; SSIM/FFT/prototype 等敏感路径升 float32。
EMA	0.999	每个验证点同时比较 raw/EMA, 由 JPG 指标选择 checkpoint 来源。
validation interval	5 epoch	每五轮跑 float、uint8、JPG 三域; 最后一轮强制验证。
primary domain	JPG	best checkpoint 首先依据更接近提交文件的 JPG 域。
inference TTA	D4 configured	已由稳定对照的全 val/5 ROI 证据确认, 提交默认八次变换。

9.4 GPU 加速选项与边界

当前配置为 CUDA 训练打开了 `channels_last`、cuDNN benchmark, 并把 float32 matmul precision 设为 `high`。固定 256×256 尺寸适合 cuDNN 自动选择卷积算法。训练器和验证器会把模型与输入转换为 channels-last 内存格式; 当前 inferencer 没有同样的显式转换, 因此不能笼统声称“训练、验证、推理全流程都使用 NHWC”。异步 metric logging、fused AdamW 和 GPU monitor 已开启; `torch.compile` 与 profiler 仍关闭, 因为前者尚未完成 eager/compiled checkpoint 回归, 后者会给正式训练增加诊断开销。网络中的“并行”表示局部、频率、邻域多路特征在同一计算图中由 GPU 批量 kernel 执行, 不等于手写多个 CUDA stream; 单卡 autograd 强行多 stream 往往会增加同步和显存。

9.5 当前通道数的一个重要事实

官方图片的 JPEG storage mode 是 RGB；审计又证明三通道逐像素相同，所以它在信息意义上是灰度图。当前 `resolved_local.yaml` 明确把输入和四个 `target` 的逻辑通道设为 1。`load_image_tensor()` 读取 RGB 后对三个通道取均值并四舍五入，模型因此构造为 **1 通道输入、1 通道输出**，参数量与 MAC 统计也以此为准。checkpoint 的 `ImageSpec` 仍记录原始 RGB storage，保存时把单通道预测复制回三个相同通道；因此“逻辑单通道”与“磁盘 mode L”不是一回事。

9.6 新手修改配置的安全原则

1. 每次只改变一个主要因素，并使用新的 run ID；
2. 先跑 1–3 epoch 检查数据、loss、显存和 checkpoint，再启动最多 100 epoch 的 v4 正式训练；
3. 比较模型时固定 manifest、seed、训练预算和验证协议；
4. 不要根据 test 反复调参，因为 test 没有本地标签；
5. 不要为了“跑起来”把 val 当 test，也不要再 train/val 间混入同一 ROI；
6. 最终选择必须看 JPG round-trip 指标，而不能只看训练 loss。

10 项目文件结构与模块职责

10.1 从根目录看项目

```
project/
|-- configs/           # YAML configurations
|-- dataset/           # data only; never import it into source code
|   |-- official/{train,test}
|-- src/virtual_staining/ # installable Python package
|-- docs/essay/report.tex # the single editable technical guide/report
|-- artifacts/         # manifests and data audit artifacts
|-- outputs/           # checkpoints, validation and predictions
|-- output/pdf/        # compiled technical report PDF
|-- log/               # compact AutoDL logs and downloadable bundles
|-- pyproject.toml     # package metadata and Python dependencies
`-- README.md          # project entry point for users
```

按上传精简要求，当前工作区没有 `tests/` 目录，`.ruff\cache/` 也已清理；上面的树因此不列测试资产。本报告不声称该精简副本通过了 `pytest`。若此版本继续长期开发，应在独立开发分支恢复与当前接口匹配的回归测试，而不是把训练数据、`checkpoint` 或提交结果当成测试替代品。

源代码采用 `src-layout`（包放在 `src/` 下的 Python 工程布局）。运行 `pip install -e .` 后，Python 能从任意工作目录找到 `virtual_staining`；`-e` 表示 `editable`，修改源码后不必重新复制安装。若完全不安装项目，也可以在项目根目录设置 `PYTHONPATH=src`，但对新手更容易漏设，因此推荐 `editable install`。

10.2 包入口与公共模块

表 13: `src/virtual_staining` 顶层模块

文件	主要职责
<code>__init__.py</code>	定义包版本和公开入口，使目录成为 Python package。
<code>constants.py</code>	集中维护 DAPI、四个目标 marker、别名与文件扩展名等稳定常量。
<code>config.py</code>	读取/深度合并 YAML，解析 CLI 覆盖，严格检查字段和取值，输出 effective config。
<code>cli.py</code>	全部命令的总控制器；把数据发现、训练、验证、预测、提交和 AutoDL 日志串成可执行工作流。

10.3 数据模块

表 14: `src/virtual_staining/data` 模块

文件	主要职责
<code>activity.py</code>	计算/分箱图像活性，用于分层报告或 <code>activity-aware sampler</code> 。
<code>audit.py</code>	全量检查尺寸、mode、通道差、值域、配对、相关性和对齐，生成数据审计报告。
<code>dataset.py</code>	根据 manifest 加载 DAPI/目标，转换为 CHW float tensor，支持单/多目标及邻域字段。
<code>discovery.py</code>	自动搜索候选数据根，识别 <code>split/organ/marker</code> 层级并给候选打分。

文件	主要职责
grouped_folds.py	以真实 ROI/group 为单位构建交叉验证 fold, 防止 patch 级随机切分泄漏。
loader_helpers.py	安全构造 DataLoader、worker seed、pin-memory/prefetch 等参数。
manifest.py	配对 DAPI 与各 marker、解析 canonical key/hash/group, 生成 train/val/test CSV 和泄漏报告。
neighborhood.py	用行列坐标查找 3×3 邻居, 返回 tiles、有效 mask 与相对 offset。
roi_audit.py	检查二维网格方向、空洞、重复坐标、边界连续性以及跨 split 邻域风险。
roi_index.py	严格解析 ROI000_00_00 为 ROI、row、col, 并建立坐标索引。
samplers.py	实现均匀或活性相关采样策略。
transforms.py	对 DAPI 和标签同步执行几何增强, 对 DAPI 单独执行允许的强度增强。
__init__.py	汇总数据子包的公开符号。

10.4 模型模块

表 15: src/virtual_staining/models 模块

文件	主要职责
baseline_unet.py	轻量 Residual U-Net 基线, 用于 smoke 和回归验证。
camp_vs_v2.py	可选 v2 总模型, 连接 local encoder、global mixer、conditioning、decoder、base/detail 和 calibrator; 当前初赛 YAML 不选择它。
naf_blocks.py	LayerNorm2d、SimpleGate、NAF block 等卷积恢复基础积木。
naf_local_encoder.py	四级局部编码器, 输出 $1/2$ 至 $1/16$ 多尺度特征。
restoration_transformer.py	GPU 友好的 Restormer-lite 通道注意力和门控前馈模块。
task_organ_conditioning.py	marker/organ embedding 与零初始化 FiLM 条件调制。

文件	主要职责
<code>context_tile_encoder.py</code>	共享参数编码邻居 tile, 结合位置 offset 和有效 mask。
<code>context_fusion.py</code>	将邻域上下文以残差 FiLM 或瓶颈交叉注意力注入中心特征。
<code>lightweight_detail_unet.py</code>	四尺度大核 depthwise/gated 轻量 U 分支, 向 1, 1/2, 1/4 skip 注入零初始化细节残差。
<code>heavy_detail_unet.py</code>	v4 四尺度重型 U 分支; 高分辨率使用 depthwise/local gated block, 低分辨率组合局部、轴向大核与空洞卷积。
<code>spatial_frequency_mixer.py</code>	在 1/8 瓶颈执行 float32 FFT low/high 分解; v4 由当前图像预测受限 cutoff, 让低/高频相互调制, 并以 route-only 接口避免第三路重复携带 NAF identity。
<code>multi_route_fusion.py</code>	对 NAF、重型 U 与频率路线同时计算逐通道/逐空间 softmax gate, 并以小残差系数和两层低分辨率 SDPA 注入。
<code>hierarchical_prototypes.py</code>	多尺度 shared/marker/organ 原型、用量与 dead-prototype 诊断。
<code>prototype_mixer.py</code>	旧版余弦原型匹配核心, 供兼容模型和扩展模块复用。
<code>laplacian_decoder.py</code>	预测 coarse/detail, 并在 v3/v4 中以零初始化 residual 叠加已验证的 direct logits, 避免初始输出突变。
<code>intensity_calibrator.py</code>	学习受限的 per-target gain/bias, 校正整体强度而避免无界漂移。
<code>multi_marker_restorer.py</code>	当前有效主模型: 四尺度 NAF 编解码、重型 Detail U、自适应频率路线、多尺度三路融合、verified context、shared/task prototypes、task adapters、稳定 base/detail 与三尺度输出。
<code>dapi_mae.py</code>	可选 DAPI masked autoencoder 预训练, 不使用标签并需 fold-local 防泄漏。
<code>registry.py</code>	按配置名构造 Residual U-Net、MultiMarker-Restorer 或 CAMP-VS v2, 并统一输出接口。
<code>__init__.py</code>	汇总模型子包接口。

10.5 损失与指标模块

表 16: 损失和指标模块

文件	主要职责
losses/charbonnier.py	平滑且较抗离群值的像素重建损失。
losses/ssim.py	可微 SSIM/MS-SSIM；敏感运算强制 float32。
losses/gradient.py	用 Sobel 梯度约束边缘结构。
losses/frequency.py	在 log-frequency 域比较纹理能量。
losses/pyramid.py	多尺度 Laplacian/pyramid 重建约束。
losses/statistics.py	约束均值、标准差等强度统计。
losses/competition_proxy.py	后段训练使用的可微 float SSIM/PSNR 代理；不冒充 uint8/JPEG round-trip。
losses/prototype.py	prototype activation/diversity 等辅助约束。
losses/shift_tolerant.py	可选小位移容忍损失；当前对齐审计为 0，默认关闭。
losses/composite.py	v1 固定权重复合损失和深监督汇总。
losses/scheduled_composite.py	v2 two-phase 权重插值与所有损失项统一入口。
metrics/image_metrics.py	逐图计算 scikit-image SSIM/PSNR。
metrics/domain_metrics.py	float、uint8、JPG 三域量化与 ROI bootstrap 差值。
metrics/aggregation.py	汇总 mean/median、ROI、target、macro、worst-10% 和 proxy。
metrics/promotion.py	按置信区间、分层退化和泄漏状态判断候选能否晋级。
各目录 __init__.py	维护稳定导入接口。

10.6 训练、推理与实验引擎

表 17: src/virtual_staining/engine 模块

文件	主要职责
common.py	公用 DataLoader、模型/损失/优化器构造和配置辅助逻辑。
optim.py	构建 no-decay 参数组与 fused AdamW，并记录实际采用的优化器实现。

文件	主要职责
<code>trainer.py</code>	训练主循环：BF16 AMP、D4 一致性、EMA、OOM microbatch、GPU 监控、时间保护、验证、early stopping 和保存。
<code>validator.py</code>	三域逐图推理、分组聚合、raw/EMA 比较和验证产物写入。
<code>inferencer.py</code>	无标签确定性预测、D4 逆变换、显存降级和 JPEG 输出。
<code>checkpoint.py</code>	保存/恢复模型、EMA、optimizer、scheduler、scaler、RNG、manifest/config hash。
<code>ema.py</code>	参数指数滑动平均的更新、state dict 与临时应用。
<code>ensemble.py</code>	多 checkpoint 预测组合与通用集成入口。
<code>ensemble_optimizer.py</code>	仅用 OOF/val 预测寻找非负权重，失败时可回退均匀平均。
<code>model_soup.py</code>	对兼容、同血缘模型做权重 soup，并检查安全前提。
<code>multistage.py</code>	管理 multitask、target、organ、metric 等多阶段训练与恢复。
<code>multitask_optimizer.py</code>	equal/FAMO 类多任务权重及梯度冲突处理实验。
<code>pretrainer.py</code>	DAPI-MAE 的 fold-local 预训练流程。
<code>experiment_registry.py</code>	记录实验父子关系、配置、状态和指标，防止选择性汇报。
<code>ablation_budget.py</code>	定义 smoke/screen/confirm/full 预算与 A0-A3 串行约束。
<code>complexity_report.py</code>	统计参数、近似 MAC 和模块占比。
<code>gradient_monitor.py</code>	监控不同任务梯度范数/夹角，诊断负迁移。
<code>prototype_diagnostics.py</code>	导出 prototype 使用率、熵、相似度等诊断。
<code>prototype_monitor.py</code>	训练期跟踪 dead/collapsed prototype，可选生成注意力图。
<code>__init__.py</code>	汇总引擎子包接口。

10.7 提交与工具模块

表 18: 提交和工具模块

文件	主要职责
<code>submission/writer.py</code>	按竞赛目录/命名复制预测, 避免二次 JPEG 编码, 并创建 ZIP。
<code>submission/validator.py</code>	校验数量、命名、mode、尺寸、像素异常、ZIP 路径和 CRC。
<code>submission/__init__.py</code>	暴露 submission 公共接口。
<code>utils/device.py</code>	发现 CPU/CUDA、显存与 BF16 能力, 解析设备和硬件 profile。
<code>utils/gpu_monitor.py</code>	周期采样 GPU 利用率、显存、功耗与温度, 按 epoch 汇总 mean/p95/max。
<code>utils/image_io.py</code>	统一 RGB/L 读取、float/uint8 转换和高质量 JPEG 保存。
<code>utils/logging.py</code>	创建 console/file logger、结构化 JSON 日志和运行元数据。
<code>utils/paths.py</code>	使用 pathlib 处理中文、空格、相对和绝对路径。
<code>utils/seed.py</code>	固定 Python、NumPy、Torch 和 DataLoader 随机种子。
<code>utils/__init__.py</code>	汇总工具子包接口。

10.8 outputs、artifacts 与 log 的区别

- artifacts/ 主要是“数据事实”：发现结果、manifest、审计、泄漏报告和校验和；
- outputs/ 主要是“模型产物”：checkpoint、验证逐图 CSV/JSON、预测和 submission；
- log/ 主要是“方便人和下一位工程师阅读的轻量记录”：终端日志、环境、GPU、吞吐、摘要及可下载 ZIP。

不要把三者混为一谈。删除 checkpoint 可以释放空间, 但 manifest 与审计文件很小且决定实验可复现性, 通常应该保留。上传给协作者做性能诊断时, 优先提供 log bundle、effective config 和 metrics, 而不是上传数 GB 的所有 checkpoint; 只有需要复现权重行为时才提供选中的 best checkpoint。

11 总结：从一张 DAPI 到竞赛 ZIP

CAMP-VS 把虚拟染色组织成一条可审计而不是“只会跑一个网络”的流水线：先发现数据并把同名 DAPI/CD68 配成 manifest; 严格按 ROI 隔离 train/val; 把 RGB storage 转换

成单逻辑通道 float tensor；当前 v4 用四尺度 NAF 主干提取稳定结构，同时由重型 Heavy Omni U-Net 恢复多尺度细胞细节、由自适应 FFT 独立路线分解并双向调制 low/high 频率，真实 3×3 DAPI 邻域提供 ROI 上下文。四个尺度使用逐通道与逐空间 softmax gate 融合，1/8 瓶颈再用两层原生 SDPA、context 和 shared/task cosine prototypes 建立全局语义；解码端以 CD68 adapter 和 zero-init base/detail residual 恢复三尺度预测；最后在 float、uint8、JPG 三个域验证 raw/EMA，并以配置化 D4 推理官方 test，进行严格 ZIP 校验。

项目当前有三个特别重要的边界。第一，本地 val 有 CD68 真值，所以能够计算 SSIM/PSNR；官方 test 只有 DAPI，本地不能算分。第二，文件存储为 RGB，但当前模型实际使用 1 输入/1 输出逻辑通道，提交时才恢复 RGB storage。第三，v4 的重型 U、自适应 spatial/frequency、多尺度三路融合、两层低分辨率 SDPA、verified context、cross-attention、prototype、task adapter 和 residual base/detail 确实进入 forward；完整 Restormer/Mamba、diffusion、GAN、calibrator、FAMO 与 shift-tolerant 等替代模块仍关闭。报告不会把“代码存在”写成“默认已使用”。

对初学者而言，最重要的实验习惯仍是保证比较公平：固定 split、seed、训练预算、三域协议和 JPEG 参数；保存 effective config、GPU/模块日志和逐图指标。v4 是“单次重训的候选组合”，不是已证明上涨的结果。只有它的 D4 JPG 指标稳定超过固定对照 0.809280/26.19677、各 ROI/活动度/边界分层没有集中退化，且平台分数也提高后，才能声称性能升级成功。

A AutoDL 从上传到训练的保姆级流程

本附录假设项目已上传到 `/root/autodl-tmp/project`，官方数据位于项目内 `dataset/official/train` 和 `dataset/official/test`，环境名为 MEDICAL。如果实际项目路径不同，只改第一条 cd；不要修改 Python 源码里的路径。

A.1 第 0 步：确认上传目录

```
cd /root/autodl-tmp/project
pwd
ls
ls dataset/official/train/DAPI | head
ls dataset/official/test/DAPI | head
```

根目录至少应看到 `pyproject.toml`、`src`、`configs`、`dataset`。train 下应有 DAPI 和 CD68 等 marker；test 下只有 DAPI 是正常的。

A.2 第 1 步：激活环境并让 Python 找到项目

推荐 editable install:

```
conda activate MEDICAL
python -m pip install -e . --no-deps
```

-e 不会复制一份大型项目，只在环境中登记源码位置；--no-deps 表示依赖已经装好，不再下载 PyTorch 等大包。若确实不愿登记项目，每次新终端都必须执行：

```
conda activate MEDICAL
export PYTHONPATH="$PWD/src:$PYTHONPATH"
```

然后验证导入和 GPU：

```
unset OMP_NUM_THREADS
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1
python -c "import virtual_staining; print('package ok')"
python -c "import torch; print(torch.__version__, torch.version.cuda); \
print('cuda=', torch.cuda.is_available()); \
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')"
```

若 cuda=False，不要启动 100 epoch。先解决 PyTorch/CUDA 环境，否则模型会落到 CPU，训练可能从数小时变成数天。

A.3 第 2 步：可选的 3 epoch 流程检查

第一次上传建议先用独立 run ID 做快速检查：

```
python -m virtual_staining.cli --log-root log autodl-run \
--config configs/initial_round_cd68_max_v4.yaml \
--data-root AUTO \
--target CD68 \
--run-id cd68_max_v4_quickcheck \
--max-epochs 3
```

终端会显示 tqdm 进度条和 epoch/batch 进度。这个命令会真实读取、审计、训练、保存和验证，但 3 epoch 结果只证明流程打通。它既不是正式成绩，也不能取代完整训练。

A.4 第 3 步：v4 正式训练与验证

使用一个新的、永不复用的 run ID：

```
python -m virtual_staining.cli --log-root log autodl-run \
  --config configs/initial_round_cd68_max_v4.yaml \
  --data-root AUTO \
  --target CD68 \
  --run-id cd68_max_v4_seed2026
```

一条命令依次执行环境记录、数据发现、manifest、ROI 邻域审计、最多 100 epoch 训练、raw/EMA 三域验证和最终 D4 验证。v4 每 5 轮验证，15% batch 执行训练期等变分支，训练阶段有 8.5 小时预测式安全停点。基于 v3 远程吞吐与 v4 75.997 GMAC 的工程估算是后段约 4.5–5.2 分钟/轮，整条命令目标 9 小时左右，但尚未在 AutoDL 完整实测。不要关闭 SSH 页面后误以为进程一定继续；若需要断线运行，应在 AutoDL 的 tmux 中执行，或使用平台提供的后台任务。

实时观察 GPU 和日志可另开一个终端：

```
watch -n 2 nvidia-smi
find log/cd68_max_v4_seed2026 -maxdepth 3 -type f
```

具体日志文件名以该 run 的 log/ 目录实际内容为准；find log -maxdepth 3 -type f 可列出全部。如果 GPU 利用率偶尔下降但进度条仍前进，可能正在做 JPEG 验证或加载数据，并不等于卡死。长时间为 0% 且 batch 不增长时，再检查日志末尾和系统内存。

A.5 第 4 步：读取验证分数

完整训练结束后，先查看：

```
python -c "import json; \
m=json.load(open('outputs/initial_round_max_v4/cd68_max_v4_seed2026/validation/ \
  metrics.json')); \
j=m['tta_comparison']['d4']['domains']['jpg']['macro']; \
print('D4 JPG SSIM =', j['mean_ssim']); \
print('D4 JPG PSNR =', j['mean_psnr']); \
print('fixed baseline = 0.809280 / 26.19677')"
```

这一步使用 official train 中隔离出的 val; 输入和 CD68 真值都存在，因此可以算分。以后改模型时，用相同 manifest、seed、训练预算和这条读取方法比较；新版本 JPG SSIM/PSNR 变高是“本地证据”，还需要逐图和 ROI 分层确认没有集中退化。它并不等于排行榜一定上升。

如需对某个 checkpoint 单独重跑验证，可执行：

```
python -m virtual_staining.cli --log-root log validate \
  --config configs/initial_round_cd68_max_v4.yaml \
  --data-root AUTO \
```

```
--checkpoint outputs/initial_round_max_v4/cd68_max_v4_seed2026/checkpoints/
best_ssim.ckpt
```

A.6 第 5 步：对 test 推理并生成提交包

最稳妥的做法是显式写出刚才选中的 checkpoint:

```
python -m virtual_staining.cli --log-root log autodl-submit \
--config configs/initial_round_cd68_max_v4.yaml \
--data-root AUTO \
--target CD68 \
--checkpoint outputs/initial_round_max_v4/cd68_max_v4_seed2026/checkpoints/
best_ssim.ckpt
```

也可以不写 checkpoint, 让程序在 `outputs/**/best_ssim.ckpt` 中按修改时间自动选最新文件:

```
python -m virtual_staining.cli --log-root log autodl-submit \
--config configs/initial_round_cd68_max_v4.yaml \
--data-root AUTO \
--target CD68
```

当前 `build_parser()` 没有给 `autodl-submit` 注册 `--run-id` 参数, 所以不要照旧版说明在提交命令末尾添加它, 否则 `argparse` 会直接报告 “unrecognized arguments”。自动选择在有多个实验时可能拿到错误权重, 因此正式提交推荐第一种显式 checkpoint 写法。提交输出目录由 checkpoint 的父 run 目录决定。

test 没有 CD68 真值, 因此命令只生成预测并检查格式, 不会显示真实 SSIM/PSNR。最终上传:

```
outputs/initial_round_max_v4/cd68_max_v4_seed2026/submission/submission_CD68.zip
```

竞赛平台用隐藏标签评分后, 返回的才是 official test score。

A.7 第 6 步：下载哪些文件

表 19: 建议从 AutoDL 下载的产物

文件	用途
<code>log/<run>/...log_bundle.zip</code>	首选诊断包; 包含环境、配置、阶段耗时、GPU/资源、指标和日志摘要。

文件	用途
outputs/.../validation/metrics.json	三域汇总指标。
outputs/.../validation/per_image.csv	找最差样本、ROI 和亮度分层退化。
outputs/.../pipeline_report.json	一次流水线的完成状态和关键路径。
outputs/.../checkpoints/best_ssim.ckpt	继续训练或重新推理需要;文件较大,不做诊断时可不传给协作者。
outputs/.../submission/submission_CD68.zip	上传竞赛平台的最终包。

如果要交给另一位工程师阅读并改进,至少提供日志压缩包、汇总/逐图指标、effective config 和平台分数截图/文本。没有这些 provenance,只给一句“跑了 15 小时分数不高”,很难区分瓶颈来自数据加载、模型、loss、验证编码还是环境。

B 常见问题与排错顺序

表 20: 新手常见故障

现象	排查与处理
No module named virtual_staining	确认位于项目根目录,执行 <code>pip install -e . --no-deps</code> , 或正确设置 <code>PYTHONPATH="\$PWD/src:\$PYTHONPATH"</code> 。
CUDA availability 为 False	检查租用实例是否有 GPU、驱动、PyTorch CUDA wheel; 修好前不要 CPU 硬跑正式训练。
CUDA out of memory	训练器先自动把 v4 物理 batch 6 切成更小 chunk, effective batch 不变; 仍失败时用 <code>--set train.batch_size=3 --set train.gradient_accumulation=4</code> 保持有效 batch 12, 再不够才开启 heavy-U activation checkpoint。
进度条似乎不动	看 <code>nvidia-smi</code> 、系统内存、日志末尾; 数据审计不是训练 batch。验证虽没有逐图 tqdm, 但会输出 raw/EMA 或 D4 pass 的开始/完成、样本数和耗时; 若连 pass 开始信息也没有, 再排查进程。
DataLoader worker 崩溃	Linux 先减少 <code>data.num_workers</code> ; Windows 本地调试设为 0。检查共享内存、损坏图和 OOM, 而非盲目重启。
JPG 指标低于 float	量化和 JPEG 会损失细节, 这是三域评估的目的; 确认质量 100、subsampling 0 且没有二次编码。

现象	排查与处理
val 很好、平台分数差	检查 ROI 分布差异、过拟合、submission target/命名、raw/EMA/TTA 选择以及本地指标与官方公式差异。
run ID 已存在	使用新的唯一 run ID；不要覆盖旧 run 后再拿混合文件做比较。
磁盘迅速增大	保留 best checkpoint、日志和指标，清理失败/重复 run 的大 checkpoint 前先核对路径；不要删除官方数据和唯一 best 权重。

C 实现事实核对清单

表 21 把容易混淆的说法与当前代码真实状态并列，便于以后代码升级后重新审阅报告。

表 21: 截至本报告生成时的实现事实

常见说法	当前真实状态
“训练集 6296 张”	6296 是每个 marker 的总配对数；当前 manifest 为 5004 train、1292 val。
“测试集可计算 SSIM”	错。1346 张 test 只有 DAPI，真实标签在平台。
“图像是 L 单通道”	storage 仍为 RGB；resolved-local 让模型实际使用 1 入/1 出逻辑通道，提交时恢复 RGB。
“v4 用了 3×3 邻域”	对。只接受 filename 坐标和 verified grid；train/val ROI 与邻接边均不重叠。
“默认用了 prototype”	对。当前启用 8 个 shared 与 8 个 CD68 task prototypes，并有两项 0.001 正则。
“NAF 模型完全没有任何激活”	NAF block 的核心是 activation-free；全模型其他分支仍可包含 GELU 等操作。
“EMA 一定比 raw 好”	错。每个验证点同时评分，由 JPG 指标选择；短训练 EMA 可能滞后。
“验证只按 YAML domains 列表算一域”	当前 Validator 实际总是生成 float、uint8、JPG 三域。
“JPG 域等于官方评分”	它最接近提交文件，但官方公式/隐藏处理不在本地代码中，不能宣称完全一致。
“普通 metrics.json 含 bootstrap CI”	普通验证有 ROI 聚合；成对 CI 在 promotion/TTA 比较路径生成。

常见说法	当前真实状态
“v4 一定跑满 100 epoch”	错。每 5 epoch 验证；5 个验证事件无改进或预测将超过 8.5 小时安全停点都可提前停止。
“所有阶段都用了 channels-last”	Trainer/Validator 显式使用；当前 Inferencer 没有同样的显式转换。
“v4 是一个完整 Restormer 或 Mamba”	错。它是 29,130,894 参数、约 75.997 GMAC 的 NAF、重型 U、自适应 FFT 与 context 三路模型；只有 1/8 瓶颈的融合器使用两层原生 SDPA，没有完整 Restormer/Mamba forward。
“base/detail 会随机替换旧输出头”	v3/v4 都不会。direct logits 保留，coarse/detail 只作为零初始化 residual，初始 prediction 与 direct head 逐位一致。
“训练 loss 里的 metric proxy 等于 JPG”	错。它只对齐 float SSIM/PSNR；uint8/JPEG round-trip 只在验证阶段真实执行。
“OOM 后 accumulation 会自动翻倍”	错。v4 只切分同一 DataLoader batch，并在必要时重启累积窗口，保持配置 effective batch。

参考文献

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *Medical Image Computing and Computer-Assisted Intervention*, 2015. <https://arxiv.org/abs/1505.04597>
- [2] L. Chen, X. Chu, X. Zhang, and J. Sun, “Simple Baselines for Image Restoration,” *European Conference on Computer Vision*, 2022. <https://arxiv.org/abs/2204.04676>
- [3] S. W. Zamir et al., “Restormer: Efficient Transformer for High-Resolution Image Restoration,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. https://openaccess.thecvf.com/content/CVPR2022/html/Zamir_Restormer_Efficient_Transformer_for_High-Resolution_Image_Restoration_CVPR_2022_paper.html
- [4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. <https://doi.org/10.1109/TIP.2003.819861>
- [5] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations*, 2019. <https://arxiv.org/abs/1711.05101>

- [6] PyTorch Contributors, “AdamW—PyTorch Documentation.” <https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW>
- [7] PyTorch Contributors, “Automatic Mixed Precision Package—PyTorch Documentation.” <https://docs.pytorch.org/docs/stable/accelerator/amp.html>
- [8] Thermo Fisher Scientific, “DAPI Stain.” <https://www.thermofisher.com/us/en/home/life-science/cell-analysis/fluorophores/dapi-stain.html>
- [9] The Human Protein Atlas, “CD68 tissue expression and primary data.” <https://www.proteinatlas.org/ENSG00000129226-CD68/tissue/primary+data>
- [10] R. Gottfried et al., “Expression of CD68 in non-myeloid cell types,” *Scandinavian Journal of Immunology*, vol. 67, no. 5, pp. 453–463, 2008. <https://pubmed.ncbi.nlm.nih.gov/18405323/>
- [11] D. L. Farber, N. A. Yudanin, and N. P. Restifo, “Human memory T cells: generation, compartmentalization and homeostasis,” *Nature Reviews Immunology*, vol. 14, pp. 24–35, 2014. <https://pmc.ncbi.nlm.nih.gov/articles/PMC4032067/>
- [12] L. J. Stern and J. M. Calvo-Calle, “HLA-DR: molecular insights and vaccine design,” *Current Pharmaceutical Design*, vol. 15, no. 28, pp. 3249–3261, 2009. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3615543/>
- [13] The Human Protein Atlas, “VIM protein expression summary.” <https://www.proteinatlas.org/ENSG00000026025-VIM>
- [14] J. C. Waters, “Accuracy and precision in quantitative fluorescence microscopy,” *Journal of Cell Biology*, vol. 185, no. 7, pp. 1135–1148, 2009. <https://pmc.ncbi.nlm.nih.gov/articles/PMC2712964/>
- [15] Y. Cui et al., “Revitalizing Convolutional Network for Image Restoration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. <https://github.com/c-yn/ConvIR>
- [16] G. Feijoo et al., “DarkIR: Robust Low-Light Image Restoration,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. <https://github.com/cidautai/DarkIR>
- [17] D. Lee et al., “ESC: Emulating Self-Attention with Convolution for Efficient Image Super-Resolution,” *IEEE/CVF International Conference on Computer Vision*, 2025. <https://github.com/dslisleedh/ESC>

- [18] Y. Cui et al., “Omni-Kernel Network for Image Restoration,” *AAAI Conference on Artificial Intelligence*, 2024. <https://github.com/c-yn/OKNet>
- [19] Y. Cui et al., “AdaIR: Adaptive All-in-One Image Restoration via Frequency Mining and Modulation,” *International Conference on Learning Representations*, 2025. <https://github.com/c-yn/AdaIR>
- [20] Y. Bai et al., “A Regularization-Guided Equivariant Approach for Image Restoration,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. [CVPR 2025 open-access paper](#)